MIST_Jackdaw Team Notebook

Md Saif Ahmed, Shihab Ahmed,
Md Rafsan Hasan Ratul

Contents

1 000 1
1.1 Remember [9 lines] - 575b8cdd . . . 1

2 Data Structure 1
2.1 DSU with Rollback [32 lines] - 3a7a732a . . . 1
2.2 DSU [35 lines] - bb29be8f . . . 2
2.3 Fenwick Tree [46 lines] - 5be59242 . . . 2
2.4 Implicit Treap [76 lines] - fd79d998 . . . 2
2.5 Mo's Algorithm [25 lines] - 4c1618b3 . . . 2
2.6 Persistent Segment Tree [42 lines] - 706b9f66 . . . 3
2.7 SQRT Decomposition [68 lines] - 1f6ad11f . . . 3
2.8 Segment Tree Lazy [86 lines] - eb2f3789 . . . 3
2.9 Segment Tree [34 lines] - 75287501 . . . 4
2.10 Sparse Table [19 lines] - ea2915a8 . . . 4
2.11 Sqrt Tricks [8 lines] - 4f4df416 . . . 4
2.12 Trie Bit [39 lines] - 53f64825 . . . 4

3 Dynamic Programming 4
3.1 Bitmask DP (TSP) [23 lines] - 626ed099 . . . 4
3.2 CHT [56 lines] - 62ef6b78 . . . 4
3.3 Digit DP [23 lines] - 0f7cf36e . . . 5
3.4 Divide and Conquer DP [28 lines] - 0a1a6cc2 . . . 5
3.5 Knapsack [23 lines] - b6337d8a . . . 5
3.6 Knuth Optimization [21 lines] - 6300eb90 . . . 5
3.7 LCIS o(n*m) [15 lines] - 4cc3ae4b . . . 5
3.8 LCS [63 lines] - a5a57237 . . . 5
3.9 LIS O(nlogn) with full path [50 lines] - 9239a87a . . . 6
3.10 Li Chao Tree [48 lines] - 03dcf00f . . . 6
3.11 MCM [24 lines] - fdf2c6cf . . . 6
3.12 SOS DP [30 lines] - 54951dc1 . . . 6

4 Game Theory 6
4.1 Points to be noted [14 lines] - f960ca72 . . . 6

5 Geometry 7
5.1 Geometry Kit [213 lines] - 2cd03012 . . . 7

6 Graph 8
6.1 0-1 BFS [18 lines] - f5b209de . . . 8
6.2 2-SAT [48 lines] - 0a20c8b4 . . . 8
6.3 Articulation Bridges [40 lines] - f1f88788 . . . 8
6.4 Articulation Points [39 lines] - 50d23cbe . . . 8
6.5 Bellmanford [40 lines] - 928d2159 . . . 8
6.6 Block Cut Tree [76 lines] - a64cd41b . . . 9
6.7 Centroid Decomposition [30 lines] - 2fb8ca4f . . . 9
6.8 DSU on Tree (Small to Large) [38 lines] - ed577778 . . . 9
6.9 Dijkstra [20 lines] - 1fbb9433 . . . 9
6.10 Dinic Max Flow [49 lines] - 974cda68 . . . 10
6.11 Euler Path (Hierholzer) [30 lines] - 65d66715 . . . 10
6.12 Euler Tour (Tree Flatten) [13 lines] - 7bf8c312 . . . 10
6.13 Floyed Warshall [13 lines] - 32069763 . . . 10
6.14 Graph Basics [55 lines] - 251fa6c0 . . . 10
6.15 Heavy Light Decomposition [80 lines] - 4efd2fd3 . . . 10
6.16 Hopcroft-Karp Matching [49 lines] - 01d1a1be . . . 11
6.17 K'th Shortest path [19 lines] - 5689f9fe . . . 11
6.18 Kruskal [14 lines] - a44dd888 . . . 11
6.19 LCA [43 lines] - 917fd387 . . . 11
6.20 Min Cost Max Flow [42 lines] - 82bfb9f8 . . . 11
6.21 SCC [49 lines] - e4c242f2 . . . 12
6.22 Topological Sort [32 lines] - f160c26e . . . 12
6.23 Tree Rerooting [94 lines] - 2f321ddc . . . 12

7 Misc 13
7.1 Bit hacks [16 lines] - df69591c . . . 13
7.2 Bitset C++ [14 lines] - e54c44bb . . . 13
7.3 Random Generator [7 lines] - c0453194 . . . 13
7.4 Special DS [25 lines] - 0c66ecd2 . . . 13
7.5 Template [22 lines] - fb722b4f . . . 13
7.6 Usefull Things [9 lines] - c67addfa . . . 13
7.7 VScode setup [7 lines] - 3e225a41 . . . 13
7.8 stress [25 lines] - 4a84e7fd . . . 13
7.9 vimrc [12 lines] - 1f47305c . . . 13

8 Number Theory 13
8.1 10-ary to m-ary [15 lines] - 116e887b . . . 13
8.2 Basis Vector [90 lines] - e1218025 . . . 14
8.3 Binary Exponentiation [12 lines] - 7916ccff . . . 14
8.4 CRT [20 lines] - f680196e . . . 14
8.5 Count 1s 0 to n [9 lines] - 9292bf04 . . . 14
8.6 Divisors multiple info [26 lines] - a222a3d7 . . . 14
8.7 Extended GCD [12 lines] - 294ded67 . . . 14
8.8 GaussElimination [40 lines] - 3f353a27 . . . 14
8.9 Lagrange Interpolation [21 lines] - b65ceb53 . . . 15
8.10 Largest Power of n! [10 lines] - 003e5b0e . . . 15
8.11 Leap Year [15 lines] - 4af3cf4a . . . 15
8.12 Linear Sieve (SPF, Phi, Mobius) [35 lines] - edb0484a . . . 15
8.13 Matrix [46 lines] - f0928c7e . . . 15
8.14 Miller-Rabin-Pollard-Rho [55 lines] - 212eb3a0 . . . 15
8.15 Mod Inverse [11 lines] - fb916c29 . . . 16
8.16 Modint [34 lines] - b857de88 . . . 16
8.17 NOD of n! [13 lines] - 60fcf48f . . . 16
8.18 NTT and FFT [85 lines] - 83a7fe8f . . . 16
8.19 No of Digits in n! in base B [7 lines] - 9ab8b6e6 . . . 16
8.20 Principle of Inclusion Exclusion [19 lines] - bd41e547 . . . 16
8.21 Print Calander of Any Year [38 lines] - 0fdba063 . . . 17
8.22 SOD, NOD Upto N [25 lines] - d2b85209 . . . 17
8.23 Segmented Sieve [16 lines] - 88be0e97 . . . 17
8.24 Sieve upto 1e9 [75 lines] - dc302c09 . . . 17
8.25 Sieve [12 lines] - 66842211 . . . 17
8.26 String Mod [6 lines] - 2b0c0c9d . . . 17
8.27 nCr [23 lines] - 8294a19d . . . 17

9 Some Algorithms 18
9.1 2D Prefix Sum [4 lines] - 3c8700da . . . 18
9.2 Big Integer [100 lines] - 72f848e2 . . . 18
9.3 Expression Parsing [90 lines] - e4cc53dd . . . 18
9.4 Kadanes [9 lines] - 8b61706c . . . 19
9.5 Monotonic Stack (next greater) [8 lines] - c63e64cb . . . 19
9.6 Ternary Search [10 lines] - c9abc776 . . . 19

10 String 19
10.1 AhoCorasick [56 lines] - ce3475dc . . . 19
10.2 Hashing [51 lines] - 05c75f15 . . . 19
10.3 KMP [29 lines] - 6273b948 . . . 19
10.4 Suffix Array [142 lines] - abba9017 . . . 20
10.5 Suffix Automaton [132 lines] - 9e196d73 . . . 20
10.6 Trie [43 lines] - 147df034 . . . 21
10.7 Z-function and Manacher [36 lines] - c59c0f88 . . . 21

11 Random 21
11.1 Combinatorics . . . 21
11.1.1 Catalan Number . . . 21
11.1.2 Stirling Number of the First Kind . . . 22
11.1.3 Stirling Numbers of the Second Kind . . . 22
11.1.4 Bell Number . . . 22
11.1.5 Lucas Theorem . . . 22
11.1.6 Derangement . . . 22
11.1.7 Burnside Lemma . . . 23
11.1.8 Eulerian Number . . . 23
11.2 Number Theory . . . 23
11.2.1 Mobius Function and Inversion . . . 23
11.2.2 GCD and LCM . . . 23
11.2.3 Gauss Circle Theorem . . . 23
11.2.4 Pick's Theorem . . . 23
11.2.5 Formula Cheatsheet . . . 23
11.2.6 Geometry and Formulas . . . 23
11.2.7 Most Number of Divisors . . . 24
11.2.8 Pascal's Triangle . . . 24
11.2.9 Logarithm Rules . . . 24
11.2.10 Miscellaneous . . . 24
11.3 Probability and Expectation . . . 25

1 000

1.1 Remember [9 lines] - 575b8cdd

```
# Don't stuck in one approach
# Bruteforce is sometimes the best way to go
# Reset global variables
# Check for overflow
# Read the problem carefully
# Break the problem into smaller parts (go step by step)
# Check for negative numbers, zero, one or small cases
# For problems like making string A to B, try to work
    backwards from B to A or it may be easier to make
    both A and B to some C
# L > R can occur
```

2 Data Structure

2.1 DSU with Rollback [32 lines] - 3a7a732a

```
// Union by size, NO path compression -> find O(log n).
// rollback(snap) undoes all unites after snapshot().
// For: offline dynamic connectivity (segtree on time),
// Mo's + DSU, divide & conquer over queries.
struct DSURollback {
  vector<int> par, sz;
  vector<pair<int, int>> ops; // {kept root, merged}
  int comps;
  DSURollback(int n)
```

```
    : par(n + 1), sz(n + 1, 1), comps(n) {
    iota(par.begin(), par.end(), 0);
  }
  int find(int u) {
    while (u != par[u]) u = par[u];
    return u;
  }
  bool unite(int u, int v) {
    u = find(u); v = find(v);
    if (u == v) return false;
    if (sz[u] < sz[v]) swap(u, v);
    par[v] = u; sz[u] += sz[v]; comps--;
    ops.push_back({u, v});
    return true;
  }
  int snapshot() { return ops.size(); }
  void rollback(int snap) {
    while ((int)ops.size() > snap) {
      auto [u, v] = ops.back(); ops.pop_back();
      par[v] = v; sz[u] -= sz[v]; comps++;
    }
  }
};
```

2.2 DSU [35 lines] - bb29be8f

```
struct DSU {
  vector<int> par, sz, minElmt, maxElmt, cntElmt;
  int cc;
 public:
  DSU(int n) {
    par.resize(n + 1);
    sz.resize(n + 1, 1);
    minElmt.resize(n + 1);
    maxElmt.resize(n + 1);
    cntElmt.resize(n + 1, 1);
    cc = n;
    for (int i = 1; i <= n; i++)
      par[i] = minElmt[i] = maxElmt[i] = i;
  }
  int find(int u) {
    if (u == par[u]) return u;
    return par[u] = find(par[u]);
  }
  void unite(int u, int v) {
    int pU = find(u);
    int pV = find(v);
    if (pU == pV) return;
    if (sz[pU] < sz[pV]) swap(pU, pV);
    par[pV] = pU;
    sz[pU] += sz[pV];
    cc--;
    cntElmt[pU] += cntElmt[pV];
    minElmt[pU] = min(minElmt[pU], minElmt[pV]);
    maxElmt[pU] = max(maxElmt[pU], maxElmt[pV]);
  }
  // min / max / count of elements in u's set
  int getMin(int u) { return minElmt[find(u)]; }
  int getMax(int u) { return maxElmt[find(u)]; }
  int getCnt(int u) { return cntElmt[find(u)]; }
};
```

2.3 Fenwick Tree [46 lines] - 5be59242

```
// 1-indexed. point add, prefix / range sum. O(log n)
struct BIT {
  int n; vector<ll> t;
  BIT(int n) : n(n), t(n + 1, 0) {}
  void update(int i, ll v) {
    for (; i <= n; i += i & -i) t[i] += v;
  }
  ll query(int i) {
    ll s = 0;
    for (; i > 0; i -= i & -i) s += t[i];
    return s;
  }
  ll query(int l, int r) {
    return query(r) - query(l - 1);
  }
  // smallest i with prefix sum >= k (all values >= 0)
  int lower_bound(ll k) {
    int pos = 0;
    for (int pw = __lg(n); pw >= 0; pw--)
      if (pos + (1 << pw) <= n &&
          t[pos + (1 << pw)] < k)
        pos += 1 << pw, k -= t[pos];
    return pos + 1;
  }
};
// Range add, point query: update(l,v), update(r+1,-v)
struct Fenwick2D {
  int n, m;
  vector<vector<ll>> t;
  Fenwick2D(int n, int m)
      : n(n), m(m), t(n + 1, vector<ll>(m + 1, 0)) {}
  void update(int i, int j, ll v) {
    for (int x = i; x <= n; x += x & -x)
      for (int y = j; y <= m; y += y & -y) t[x][y] += v;
  }
  ll query(int i, int j) {
    ll s = 0;
    for (int x = i; x > 0; x -= x & -x)
      for (int y = j; y > 0; y -= y & -y) s += t[x][y];
    return s;
  }
  ll query(int r1, int c1, int r2, int c2) {
    return query(r2, c2) - query(r1 - 1, c2)
         - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
  }
};
```

2.4 Implicit Treap [76 lines] - fd79d998

```
// Array with split / merge in O(log n) expected:
// insert / erase at position, range reverse, sum.
// Positions are 0-indexed. Edit pull() / push() for
// other range ops (min, add lazy, ...)
mt19937 rngT(chrono::steady_clock::now()
                .time_since_epoch().count());
struct Treap {
  struct Node {
    int l = 0, r = 0, sz = 1; unsigned pri;
    ll val, sum; bool rev = false;
  };
  vector<Node> t = {Node()}; // t[0] = null, sz 0
  Treap() { t[0].sz = 0; t[0].val = t[0].sum = 0; }
  int newNode(ll v) {
    Node x; x.pri = rngT(); x.val = x.sum = v;
    t.push_back(x); return t.size() - 1;
  }
  void push(int u) {
    if (u && t[u].rev) {
      swap(t[u].l, t[u].r);
      if (t[u].l) t[t[u].l].rev ^= 1;
      if (t[u].r) t[t[u].r].rev ^= 1;
      t[u].rev = false;
    }
  }
  void pull(int u) {
    t[u].sz = 1 + t[t[u].l].sz + t[t[u].r].sz;
    t[u].sum = t[u].val + t[t[u].l].sum
             + t[t[u].r].sum;
  }
  // first k elements -> a, rest -> b
  void split(int u, int k, int& a, int& b) {
    if (!u) { a = b = 0; return; }
    push(u);
    if (t[t[u].l].sz < k) {
      split(t[u].r, k - t[t[u].l].sz - 1, t[u].r, b);
      a = u;
    } else {
      split(t[u].l, k, a, t[u].l);
      b = u;
    }
    pull(u);
  }
  int merge(int a, int b) {
    if (!a || !b) return a ? a : b;
    if (t[a].pri > t[b].pri) {
      push(a); t[a].r = merge(t[a].r, b);
      pull(a); return a;
    }
    push(b); t[b].l = merge(a, t[b].l);
    pull(b); return b;
  }
  int root = 0;
  void insert(int pos, ll v) { // new elem at pos
    int a, b; split(root, pos, a, b);
    root = merge(merge(a, newNode(v)), b);
  }
  void erase(int pos) {
    int a, b, c; split(root, pos, a, b);
    split(b, 1, b, c); root = merge(a, c);
  }
  // apply f to the treap of range [l, r]
  template <class F> void range(int l, int r, F f) {
    int a, b, c; split(root, l, a, b);
    split(b, r - l + 1, b, c);
    f(b); root = merge(merge(a, b), c);
  }
  void reverse(int l, int r) {
    range(l, r, [&](int u) { if (u) t[u].rev ^= 1; });
  }
  ll sum(int l, int r) {
    ll s = 0;
    range(l, r, [&](int u) { s = t[u].sum; });
    return s;
  }
};
```

2.5 Mo's Algorithm [25 lines] - 4c1618b3

```
// Offline range queries, O((n + q) sqrt n * add cost)
// Edit add / del / answer. Queries 0-indexed [l, r]
struct Query { int l, r, idx; };
const int BLK = 450; // ~ n / sqrt(q)
int cnt[1000005], cur = 0; // example: # distinct
vector<int> a;
void add(int i) { if (cnt[a[i]]++ == 0) cur++; }
void del(int i) { if (--cnt[a[i]] == 0) cur--; }
vector<int> mo(vector<Query> qs) {
  sort(qs.begin(), qs.end(), [](Query& x, Query& y) {
```

```
    if (x.l / BLK != y.l / BLK) return x.l < y.l;
    // odd-even trick: alternate r direction
    return (x.l / BLK & 1) ? x.r > y.r : x.r < y.r;
  });
  vector<int> ans(qs.size());
  int L = 0, R = -1;
  for (auto& q : qs) {
    while (R < q.r) add(++R);
    while (L > q.l) add(--L);
    while (R > q.r) del(R--);
    while (L < q.l) del(L++);
    ans[q.idx] = cur;
  }
  return ans;
}
```

2.6 Persistent Segment Tree [42 lines] - 706b9f66

```
// Persistent segtree over compressed values [0, sz).
// root[i] = tree of a[1..i]; [l,r] = root[r]-root[l-1]
// kth smallest in a[l..r], count <= x: O(log n)
const int MAXN = 2e5 + 5, LOG = 20;
int lc[MAXN * LOG], rc[MAXN * LOG], cnt[MAXN * LOG];
int nodes;
int root[MAXN];
int update(int prev, int l, int r, int pos) { // +1
  int cur = ++nodes;
  lc[cur] = lc[prev], rc[cur] = rc[prev];
  cnt[cur] = cnt[prev] + 1;
  if (l == r) return cur;
  int mid = (l + r) / 2;
  if (pos <= mid)
    lc[cur] = update(lc[prev], l, mid, pos);
  else rc[cur] = update(rc[prev], mid + 1, r, pos);
  return cur;
}
// k-th smallest (1-based) -> index in comp
// u = root[r], v = root[l-1]
int kth(int u, int v, int l, int r, int k) {
  if (l == r) return l;
  int mid = (l + r) / 2, left = cnt[lc[u]] - cnt[lc[v]];
  if (k <= left) return kth(lc[u], lc[v], l, mid, k);
  return kth(rc[u], rc[v], mid + 1, r, k - left);
}
// how many values with index <= pos
int countLE(int u, int v, int l, int r, int pos) {
  if (pos < l) return 0;
  if (r <= pos) return cnt[u] - cnt[v];
  int mid = (l + r) / 2;
  return countLE(lc[u], lc[v], l, mid, pos)
       + countLE(rc[u], rc[v], mid + 1, r, pos);
}
/* node 0 = empty tree (all zero). Usage:
comp = sorted unique a; sz = comp.size();
for i = 1..n: root[i] = update(root[i-1], 0, sz - 1,
                lower_bound(comp, a[i]) index);
k-th in [l,r]: comp[kth(root[r], root[l-1], 0, sz-1,
    k)]
#(<= x) in [l,r]: p = upper_bound(comp, x) - 1, (p < 0:
  answer 0) countLE(root[r], root[l-1], 0, sz-1, p)
*/
```

2.7 SQRT Decomposition [68 lines] - 1f6ad11f

```
// https://www.spoj.com/problems/GIVEAWAY/
// 0 l r k : count of numbers >= k in [l,r]
// 1 idx val : update a[idx] = val
const int N = 5e5 + 5, BLK = 700;
vector<int> block[N / BLK + 5];
int get_block(int idx) { return idx / BLK; }
void solve() {
  int n;
  cin >> n;
  vector<int> a(n);
  for (int i = 0; i < n; i++) cin >> a[i];
  for (int i = 0; i < n; i++) {
    block[get_block(i)].push_back(a[i]);
  }
  int last_blk = get_block(n - 1);
  for (int i = 0; i <= last_blk; i++) {
    sort(block[i].begin(), block[i].end());
  }
  int q;
  cin >> q;
  while (q--) {
    int ty;
    cin >> ty;
    if (ty == 0) {
      // query
      // O(sqrt(n) log(sqrt(n)))
      int l, r, k;
      cin >> l >> r >> k;
      l--, r--;
      int bl = get_block(l);
      int br = get_block(r);
      int ans = 0;
      if (bl == br) {
        // just brute force
        for (int i = l; i <= r; i++) {
          if (a[i] >= k) ans++;
        }
      } else {
        for (int i = l; get_block(i) < (bl + 1); i++) {
          if (a[i] >= k) ans++;
        }
        for (int i = r; get_block(i) > (br - 1); i--) {
          if (a[i] >= k) ans++;
        }
        for (int b = bl + 1; b < br; b++) {
          // sorted [1, 5, 10, 20], k = 6 -> ans += 2
          auto it = lower_bound(block[b].begin(),
                                block[b].end(), k);
          ans += block[b].end() - it;
        }
      }
      cout << ans << '\n';
    } else {
      // update
      // O(sqrt(n) log(sqrt(n)))
      int idx, val;
      cin >> idx >> val;
      idx--;
      int old = a[idx];
      int b = get_block(idx);
      auto it = lower_bound(block[b].begin(),
                            block[b].end(), old);
      *it = val;
      sort(block[b].begin(), block[b].end());
      a[idx] = val;
    }
  }
}
```

2.8 Segment Tree Lazy [86 lines] - eb2f3789

```
/* 1. TO ADAPT FOR ANY PROBLEM, ONLY EDIT THESE 4:
  - `Node` : Add your variables (sum, mx, pref, suff).
            Node() must be the identity element!
  - `combine()`  : How to merge left & right children.
  - `applySet()` : How setting a range to `v` changes a
                   Node. (Updates lzSet, clears lzAdd)
  - `applyAdd()` : How adding `v` to a range changes a
                   Node. (Updates lzAdd)
 2. NEVER TOUCH `push()`, `update()`, OR `query()`!
 3. WHY applySet CLEARS lzAdd? Overwriting a range
    completely destroys its previous history (adds).
 4. WHY push SET BEFORE ADD? "Set to X, then add Y"
    implies Set happened first.
 5. FOR TOGGLE/XOR: Change `lzAdd` to `lzXor`, delete
    `Set` logic, update `applyAdd`.
 6. 1-BASED: build(1, 1, n, a), query(1, 1, n, l, r),
    update(1, 1, n, l, r, v, type): 1 = add, else set
*/
struct Node {
  ll sum;
  Node(ll _sum = 0) { sum = _sum; }
};
struct STLazy {
  int n; vector<Node> seg;
  vector<ll> lzAdd, lzSet; vector<bool> hasSet;
  STLazy(int _n) : n(_n), seg(4*_n+5),
    lzAdd(4*_n+5, 0), lzSet(4*_n+5, 0),
    hasSet(4*_n+5, 0) {}
  Node combine(Node l, Node r) {
    Node res; res.sum = l.sum + r.sum;
    return res;
  }
  void applySet(int node, int l, int r, ll v) {
    seg[node].sum = (ll)(r - l + 1) * v;
    lzSet[node] = v; hasSet[node] = 1; lzAdd[node] = 0;
  }
  void applyAdd(int node, int l, int r, ll v) {
    seg[node].sum += (ll)(r - l + 1) * v;
    lzAdd[node] += v;
  }
  void push(int node, int l, int r) {
    if (l == r) return;
    int mid = (l + r) >> 1;
    int lc = node << 1, rc = node << 1 | 1;
    if (hasSet[node]) {
      applySet(lc, l, mid, lzSet[node]);
      applySet(rc, mid + 1, r, lzSet[node]);
      hasSet[node] = 0;
    }
    if (lzAdd[node] != 0) {
      applyAdd(lc, l, mid, lzAdd[node]);
      applyAdd(rc, mid + 1, r, lzAdd[node]);
      lzAdd[node] = 0;
    }
  }
  void build(int node, int l, int r,
             const vector<ll>& a) {
    if (l == r) { seg[node] = Node(a[l]); return; }
    int mid = (l + r) >> 1;
    build(node<<1, l, mid, a);
    build(node<<1|1, mid+1, r, a);
    seg[node] = combine(seg[node<<1], seg[node<<1|1]);
  }
  void update(int node, int l, int r, int ql, int qr,
              ll v, int type) {
    if (ql > r || qr < l) return;
```

```
    if (ql <= l && r <= qr) {
      if (type == 1) applyAdd(node, l, r, v);
      else applySet(node, l, r, v);
      return;
    }
    push(node, l, r);
    int mid = (l + r) >> 1;
    update(node<<1, l, mid, ql, qr, v, type);
    update(node<<1|1, mid+1, r, ql, qr, v, type);
    seg[node] = combine(seg[node<<1], seg[node<<1|1]);
  }
  Node query(int node, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return Node();
    if (ql <= l && r <= qr) return seg[node];
    push(node, l, r);
    int mid = (l + r) >> 1;
    return combine(query(node<<1, l, mid, ql, qr),
                   query(node<<1|1, mid+1, r, ql, qr));
  }
};
```

2.9 Segment Tree [34 lines] - 75287501

```
struct Node {
  ll sum;
  Node() { sum = 0; }
};
struct ST {
  int n; vector<Node> seg;
  ST(int _n) : n(_n), seg(4*_n+5) {}
  Node combine(Node l, Node r) {
    Node res; res.sum = l.sum + r.sum;
    return res;
  }
  void build(int node, int l, int r,
             const vector<ll>& a) {
    if (l == r) { seg[node].sum = a[l]; return; }
    int mid = (l + r) >> 1;
    build(node<<1, l, mid, a);
    build(node<<1|1, mid+1, r, a);
    seg[node] = combine(seg[node<<1], seg[node<<1|1]);
  }
  void update(int node, int l, int r, int pos, ll v) {
    if (l == r) { seg[node].sum = v; return; }
    int mid = (l + r) >> 1;
    if (pos <= mid) update(node<<1, l, mid, pos, v);
    else update(node<<1|1, mid+1, r, pos, v);
    seg[node] = combine(seg[node<<1], seg[node<<1|1]);
  }
  Node query(int node, int l, int r, int ql, int qr) {
    if (ql > r || qr < l) return Node();
    if (ql <= l && r <= qr) return seg[node];
    int mid = (l + r) >> 1;
    return combine(query(node<<1, l, mid, ql, qr),
                   query(node<<1|1, mid+1, r, ql, qr));
  }
};
```

2.10 Sparse Table [19 lines] - ea2915a8

```
// Static RMQ: O(n log n) build, O(1) query. Op must be
// idempotent: min, max, gcd, and, or. 0-indexed [l, r]
template <class T> struct Sparse {
  vector<vector<T>> st;
  T op(T a, T b) { return min(a, b); } // EDIT
  Sparse(const vector<T>& a) {
    int n = a.size(), K = __lg(max(n, 1)) + 1;
    st.assign(K, a);
    for (int k = 1; k < K; k++)
      for (int i = 0; i + (1 << k) <= n; i++)
        st[k][i] = op(st[k - 1][i],
                      st[k - 1][i + (1 << (k - 1))]);
  }
  T query(int l, int r) {
    int k = __lg(r - l + 1);
    return op(st[k][l], st[k][r - (1 << k) + 1]);
  }
};
// Sum (not idempotent): split into powers of 2
```

2.11 Sqrt Tricks [8 lines] - 4f4df416

1. Size of the block is not always Sqrt, adjust it as necessary. if o(n/b+b) then take n/b = b and calculate b.
2. MO's Algorithm
   *it is possible to solve a Mo problem without any remove operation. For L in one block R only increases, for every range we can start L from the last of that block
3. Sqrt Decomposition by time of queries.
   *keep overall solution and sqrt(n) updates in a vector and for a query iterate over all of them, when the vector size exceeds sqrt(n) you can add these updates with overall solution using o(n)
4. If sum of N positive numbers are S, there are at most sqrt(S) distinct values.
5. Randomization
6. Baby step, gaint step

2.12 Trie Bit [39 lines] - 53f64825

```
struct BitTrie {
  static const int BT = 62; // 30 for numbers <= 1e9
  struct Node {
    int nxt[2]{}; // {} zero-initializes the array
    int cnt = 0;
  };
  vector<Node> t;
  BitTrie() : t(1) {} // root at index 0
  //d = 1 for insert, d = -1 for remove
  void insert(long long x, int d = 1) {
    int u = 0;
    t[u].cnt += d; // Track total elements at root
    for (int i = BT - 1; i >= 0; i--) {
      int v = (x >> i) & 1;
      if (!t[u].nxt[v]) {
        t[u].nxt[v] = t.size();
        t.emplace_back();
      }
      u = t[u].nxt[v];
      t[u].cnt += d;
    }
  }
  long long query(long long x) {
    if (!t[0].cnt) return 0;
    int u = 0;
    long long mx = 0;
    for (int i = BT - 1; i >= 0; i--) {
      int r = (x >> i) & 1;
      if (t[u].nxt[r ^ 1] &&
          t[t[u].nxt[r ^ 1]].cnt > 0) {
        mx |= (1LL << i);
        u = t[u].nxt[r ^ 1];
      } else {
        u = t[u].nxt[r];
      }
    }
    return mx;
  }
};
```

3 Dynamic Programming

3.1 Bitmask DP (TSP) [23 lines] - 626ed099

```
// TSP: shortest cycle through all n nodes, O(2^n n^2)
// dp[mask][u]: start 0, visited = mask, now at u
ll tsp(vector<vector<ll>>& d) {
  int n = d.size();
  const ll INF = 1e18;
  vector<vector<ll>> dp(1 << n, vector<ll>(n, INF));
  dp[1][0] = 0;
  for (int mask = 1; mask < (1 << n); mask++)
    for (int v = 0; v < n; v++) {
      if (!(mask >> v & 1) || dp[mask][v] >= INF)
        continue;
      for (int u = 0; u < n; u++)
        if (!(mask >> u & 1)) {
          int nm = mask | 1 << u;
          dp[nm][u] =
              min(dp[nm][u], dp[mask][v] + d[v][u]);
        }
    }
  ll ans = INF; // for a path: min of dp[full][v]
  for (int v = 0; v < n; v++)
    ans = min(ans, dp[(1 << n) - 1][v] + d[v][0]);
  return ans;
}
```

3.2 CHT [56 lines] - 62ef6b78

```
/* INITIALIZATION & INSERTION RULES:
- Max Query: `CHT cht(true);`  --> Slopes MUST be added
  INCREASING (m1 <= m2 <= m3)
- Min Query: `CHT cht(false);` --> Slopes MUST be added
  DECREASING (m1 >= m2 >= m3)
QUERY RULES:
- `query(x)`: O(log N) for ANY x -> USE THIS BY DEFAULT
- `query_monotonic(x)`: O(1) -> ONLY if x INCREASING!
  (x decreasing: query(x) or reverse offline queries)
(Slopes in ANY order -> use Li Chao Tree) */
struct CHT {
  struct Line {
    ll m, c;
    ll eval(ll x) const { return m * x + c; }
  };
  vector<Line> lines; bool is_max; int ptr = 0;
  CHT(bool is_max = true) : is_max(is_max) {}
  bool bad(const Line& l1, const Line& l2,
           const Line& l3) {
    return (__int128)(l1.c - l2.c) * (l3.m - l2.m) >=
           (__int128)(l2.c - l3.c) * (l2.m - l1.m);
  }
  void add(ll m, ll c) {
    if (!is_max) { m = -m; c = -c; }
    Line l = {m, c};
    while (!lines.empty() && lines.back().m == l.m) {
      if (lines.back().c >= l.c) return;
      lines.pop_back();
    }
    while (lines.size() >= 2 &&
        bad(lines[lines.size() - 2], lines.back(), l))
      lines.pop_back();
    lines.push_back(l);
```

```
  }
  ll query(ll x) const {
    assert(!lines.empty());
    int l = 0, r = lines.size() - 1;
    while (l < r) {
      int mid = l + (r - l) / 2;
      if (lines[mid].eval(x) < lines[mid + 1].eval(x))
        l = mid + 1;
      else r = mid;
    }
    return is_max ? lines[l].eval(x)
                  : -lines[l].eval(x);
  }
  ll query_monotonic(ll x) {
    assert(!lines.empty());
    ptr = min(ptr, (int)lines.size() - 1);
    while (ptr + 1 < lines.size() &&
           lines[ptr].eval(x) < lines[ptr + 1].eval(x))
      ptr++;
    return is_max ? lines[ptr].eval(x)
                  : -lines[ptr].eval(x);
  }
};
```

3.3 Digit DP [23 lines] - 0f7cf36e

```
// Count x in [0, X] with a digit property. Answer for
// [L, R] = solve(R) - solve(L-1). Example: digit sum K
// tight: prefix equals X so far; started: non-zero
    seen
// (use `started` when leading zeros matter)
string s; int K;
ll dp[20][2][2][200]; // pos, tight, started, sum
ll f(int pos, bool tight, bool started, int sum) {
  if (pos == (int)s.size()) return sum == K; // EDIT
  ll &res = dp[pos][tight][started][sum];
  if (res != -1) return res;
  res = 0;
  int lim = tight ? s[pos] - '0' : 9;
  for (int d = 0; d <= lim; d++)
    res += f(pos + 1, tight && d == lim, started || d,
             sum + d);
  return res;
}
ll solve(ll X) {
  if (X < 0) return 0;
  s = to_string(X);
  memset(dp, -1, sizeof dp);
  return f(0, 1, 0, 0);
}
```

3.4 Divide and Conquer DP [28 lines] - 0a1a6cc2

```
// Split C[1..L] into G non-empty groups, min total
    cost
// dp[g][i] = min_{j<=i} dp[g-1][j-1] + cost(j, i)
// Valid when opt(g, i) <= opt(g, i+1). O(G * L log L)
const ll INF = 1e18;
int G, L;
vector<ll> pre;            // pre[i] = C[1] + ... + C[i]
vector<vector<ll>> dp;     // dp[G + 1][L + 1]
ll cost(int l, int r) { // EDIT: cost of group [l, r]
  return (pre[r] - pre[l - 1]) * (r - l + 1);
}
void calc(int g, int st, int ed, int optl, int optr) {
  if (st > ed) return;
  int mid = (st + ed) / 2, pos = optl;
  dp[g][mid] = INF;
  for (int i = optl; i <= min(mid, optr); i++) {
    if (dp[g - 1][i - 1] >= INF) continue;
    ll c = dp[g - 1][i - 1] + cost(i, mid);
    if (c < dp[g][mid]) dp[g][mid] = c, pos = i;
  }
  calc(g, st, mid - 1, optl, pos);
  calc(g, mid + 1, ed, pos, optr);
}
ll solve() {
  dp.assign(G + 1, vector<ll>(L + 1, INF));
  dp[0][0] = 0;
  for (int g = 1; g <= G; g++) calc(g, 1, L, 1, L);
  return dp[G][L];
}
```

3.5 Knapsack [23 lines] - b6337d8a

```
// dp[c] = best value with capacity c (W = capacity)
// 0/1: each item once -> iterate c DOWNWARD
for (int i = 0; i < n; i++)
  for (int c = W; c >= w[i]; c--)
    dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
// Unbounded: item many times -> iterate c UPWARD
for (int i = 0; i < n; i++)
  for (int c = w[i]; c <= W; c++)
    dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
// Bounded (item i at most k[i] times): binary split
// into items of 1, 2, 4, ..., rest copies, then 0/1.
for (int i = 0; i < n; i++)
  for (int p = 1, left = k[i]; left > 0; p <<= 1) {
    int take = min(p, left); left -= take;
    W2.push_back(take * w[i]);
    V2.push_back(take * v[i]);
  }
// Count ways to make sum S (coins, order ignored):
// ways[0] = 1; for coin: for s = coin..S:
//   ways[s] += ways[s - coin]
// (order matters: swap the loops -> s outside)
// Subset sum up to 1e5-1e6: bitset b; b[0] = 1;
//   for (x : a) b |= b << x;
```

3.6 Knuth Optimization [21 lines] - 6300eb90

```
// dp[i][j] = min_{i<=k<j} dp[i][k]+dp[k+1][j]+C(i,j)
// valid if opt[i][j-1] <= opt[i][j] <= opt[i+1][j]
// (C quadrangle inequality, e.g. merging stones /
// optimal BST). O(n^2) instead of O(n^3). 0-indexed
ll knuth(int n, function<ll(int, int)> C) {
  vector<vector<ll>> dp(n, vector<ll>(n, 0));
  vector<vector<int>> opt(n, vector<int>(n));
  for (int i = 0; i < n; i++) opt[i][i] = i;
  for (int len = 2; len <= n; len++)
    for (int i = 0; i + len - 1 < n; i++) {
      int j = i + len - 1;
      dp[i][j] = LLONG_MAX;
      int lo = opt[i][j - 1];
      int hi = min(j - 1, opt[i + 1][j]);
      for (int k = lo; k <= hi; k++) {
        ll c = dp[i][k] + dp[k + 1][j] + C(i, j);
        if (c < dp[i][j]) dp[i][j] = c, opt[i][j] = k;
      }
    }
  return dp[0][n - 1];
}
```

3.7 LCIS o(n*m) [15 lines] - 4cc3ae4b

```
// Longest Common Increasing Subsequence, O(n * m)
// f[j] = LCIS of a[0..i] and b[0..j] that ends at b[j]
int LCIS(vector<int>& a, vector<int>& b) {
  int m = b.size(), ans = 0;
  vector<int> f(m, 0);
  for (int x : a) {
    int best = 0; // best f[j] with b[j] < x so far
    for (int j = 0; j < m; j++) {
      if (b[j] < x) best = max(best, f[j]);
      else if (b[j] == x) f[j] = max(f[j], best + 1);
    }
  }
  for (int j = 0; j < m; j++) ans = max(ans, f[j]);
  return ans;
}
```

3.8 LCS [63 lines] - a5a57237

```
// LCS DP Table and LCS Length
vector<vector<int>> dp(s.size() + 1,
                       vector<int>(t.size() + 1));
for (int i = 1; i <= s.size(); i++) {
  for (int j = 1; j <= t.size(); j++) {
    if (s[i - 1] == t[j - 1])
      dp[i][j] = dp[i - 1][j - 1] + 1;
    else
      dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
  }
}
cout << dp[s.size()][t.size()] << "\n";
// Any LCS String
string a;
int i = s.size();
int j = t.size();
while (i > 0 && j > 0) {
  if (s[i - 1] == t[j - 1]) {
    a += s[i - 1];
    i--, j--;
  } else if (dp[i][j - 1] > dp[i - 1][j])
    j--;
  else
    i--;
}
reverse(a.begin(), a.end());
cout << a << "\n";
// Lexicographically smallest LCS
vector<vector<string>> dpp(s.size() + 1,
                           vector<string>(t.size() + 1));
for (int i = 1; i <= s.size(); i++) {
  for (int j = 1; j <= t.size(); j++) {
    if (s[i - 1] == t[j - 1])
      dpp[i][j] = dpp[i - 1][j - 1] + s[i - 1];
    else if (dp[i][j - 1] < dp[i - 1][j])
      dpp[i][j] = dpp[i - 1][j];
    else if (dp[i][j - 1] > dp[i - 1][j])
      dpp[i][j] = dpp[i][j - 1];
    else
      dpp[i][j] = min(dpp[i][j - 1], dpp[i - 1][j]);
  }
}
cout << dpp[s.size()][t.size()] << "\n";
// number of distinct LCS sequences
int MOD = 1e9 + 7;
vector<vector<int>> cnt(s.size() + 1,
                        vector<int>(t.size() + 1, 1));
for (int i = 1; i <= s.size(); i++) {
  for (int j = 1; j <= t.size(); j++) {
    if (s[i - 1] == t[j - 1])
      cnt[i][j] = cnt[i - 1][j - 1];
    else if (dp[i][j - 1] < dp[i - 1][j])
```

```
      cnt[i][j] = cnt[i - 1][j];
    else if (dp[i][j - 1] > dp[i - 1][j])
      cnt[i][j] = cnt[i][j - 1];
    else {
      cnt[i][j] = cnt[i - 1][j] + cnt[i][j - 1];
      if (dp[i][j] == dp[i - 1][j - 1])
        cnt[i][j] -= cnt[i - 1][j - 1];
      cnt[i][j] = (cnt[i][j] + MOD) % MOD;
    }
  }
}
```

3.9 LIS O(nlogn) with full path [50 lines] - 9239a87a

```
// 1. LIS Length (upper_bound instead: allow EQUAL)
int LIS(vector<int>& v) {
  vector<int> seq;
  for (int x : v) {
    auto it = lower_bound(seq.begin(), seq.end(), x);
    if (it == seq.end()) seq.push_back(x);
    else *it = x;
  }
  return seq.size();
}
// 2. LIS with Reconstruction
vector<int> getLIS(vector<int>& v) {
  int n = v.size(); vector<int> seq, id, p(n, -1), ans;
  for (int i = 0; i < n; i++) {
    int pos = lower_bound(seq.begin(), seq.end(), v[i])
              - seq.begin();
    if (pos > 0) p[i] = id[pos - 1]; // link to prev
    if (pos == seq.size()) {
      seq.push_back(v[i]); id.push_back(i);
    } else { seq[pos] = v[i]; id[pos] = i; }
  }
  // Backtrack using parent array `p`
  int c = id.empty() ? -1 : id.back();
  for (; c != -1; c = p[c]) ans.push_back(v[c]);
  reverse(ans.begin(), ans.end()); return ans;
}
// 3. SEGTREE LIS: Weights/Counts/Restrictions O(NlogN)
// Assumes Point Update, Range Max Query SegTree ready.
ll getLIS_SegTree(vector<int>& a, vector<ll>& w) {
  vector<int> b = a; ll ans = 0;
  // Coord Compress: leaves [1..M] = the SORTED VALUES
  sort(b.begin(), b.end());
  b.erase(unique(b.begin(), b.end()), b.end());
  int M = b.size();
  // Call build(1, 1, M) here to initialize leaves to 0
  for (int i = 0; i < a.size(); i++) {
    // x is the 1-based compressed index of value a[i]
    int x = lower_bound(b.begin(), b.end(), a[i])
            - b.begin() + 1;
    // 1. Strictly Increasing: query(1, 1, M, 1, x - 1)
    // 2. Non-Decreasing: query(1, 1, M, 1, x)
    // 3. Condition (e.g. diff >= K): find pos of
    //    (a[i]-K) in 'b' and query up to pos
    ll mx = (x == 1) ? 0 : query(1, 1, M, 1, x - 1);
    mx += w[i]; ans = max(ans, mx);
    // Update max value ending exactly at compressed x
    update(1, 1, M, x, mx);
  }
  return ans;
}
```

3.10 Li Chao Tree [48 lines] - 03dcf00f

```
// MIN query, any line order, x in [L, R]. O(log C)
// MAX: add(-m,-c), ans=-query(x). Dynamic, x up to 1e9
// add_seg(l,r,m,c): line only on [l,r]. O(log^2 C)
struct LiChao {
  struct Line {
    ll m = 0, c = LLONG_MAX;
    ll eval(ll x) const { return m * x + c; }
  };
  struct Node { Line ln; int lc = 0, rc = 0; };
  vector<Node> t; ll L, R;
  LiChao(ll L, ll R) : t(2), L(L), R(R) {} // 1=root
  int child(int u, bool r) { // make if missing
    if (!(r ? t[u].rc : t[u].lc)) {
      int x = t.size(); t.emplace_back();
      (r ? t[u].rc : t[u].lc) = x;
    }
    return r ? t[u].rc : t[u].lc;
  }
  void add(int u, ll l, ll r, Line nl) {
    ll mid = l + (r - l) / 2;
    bool lef = nl.eval(l) < t[u].ln.eval(l);
    bool md = nl.eval(mid) < t[u].ln.eval(mid);
    if (md) swap(t[u].ln, nl);
    if (l == r || nl.c == LLONG_MAX) return;
    if (lef != md) add(child(u, 0), l, mid, nl);
    else add(child(u, 1), mid + 1, r, nl);
  }
  void add_seg(int u, ll l, ll r, ll a, ll b, Line nl) {
    if (b < l || r < a) return;
    if (a <= l && r <= b) { add(u, l, r, nl); return; }
    ll mid = l + (r - l) / 2;
    add_seg(child(u, 0), l, mid, a, b, nl);
    add_seg(child(u, 1), mid + 1, r, a, b, nl);
  }
  ll query(int u, ll l, ll r, ll x) {
    if (!u) return LLONG_MAX;
    ll res = t[u].ln.eval(x), mid = l + (r - l) / 2;
    if (l == r) return res;
    ll q = x <= mid ? query(t[u].lc, l, mid, x)
                    : query(t[u].rc, mid + 1, r, x);
    return min(res, q);
  }
  void add(ll m, ll c) { add(1, L, R, {m, c}); }
  void add_seg(ll l, ll r, ll m, ll c) {
    add_seg(1, L, R, l, r, {m, c});
  }
  ll query(ll x) { return query(1, L, R, x); }
};
```

3.11 MCM [24 lines] - fdf2c6cf

```
// Matrix i has size v[i-1] x v[i], i = 1..n. O(n^3)
// memset(dp, -1, sizeof dp); MCM(1, n);
// print_order(1, n);
const int N = 505;
vector<ll> v;
ll dp[N][N]; int mark[N][N];
ll MCM(int i, int j) {
  if (i == j) return dp[i][j] = 0;
  if (dp[i][j] != -1) return dp[i][j];
  ll mn = LLONG_MAX;
  for (int k = i; k < j; k++) {
    ll c = MCM(i, k) + MCM(k + 1, j)
         + v[i - 1] * v[k] * v[j];
    if (c < mn) mn = c, mark[i][j] = k;
  }
  return dp[i][j] = mn;
}
void print_order(int i, int j) {
  if (i == j) { cout << "X" << i; return; }
  cout << "(";
  print_order(i, mark[i][j]);
  print_order(mark[i][j] + 1, j);
  cout << ")";
}
```

3.12 SOS DP [30 lines] - 54951dc1

```
// SOS DP, O(MLOG * 2^MLOG). Set dp[mask] = f(mask):
// forward1: dp[mask] = sum of f(sub) over submasks sub
// backward1: inverse of forward1 (Mobius transform)
// forward2 / backward2: same for supermasks
// All submasks of mask, O(3^n) total over all masks:
// for (int s = mask; s; s = (s-1) & mask) {} (+ s=0)
const int MAXN = 1 << 20, MLOG = 20;
ll dp[MAXN];
// Sum over subsets (SOS Down)
void forward1() {
  for (int bit = 0; bit < MLOG; bit++)
    for (int i = 0; i < MAXN; i++)
      if (i & (1 << bit)) dp[i] += dp[i ^ (1 << bit)];
}
void backward1() {
  for (int bit = 0; bit < MLOG; bit++)
    for (int i = MAXN - 1; i >= 0; i--)
      if (i & (1 << bit)) dp[i] -= dp[i ^ (1 << bit)];
}
// Sum over supersets (SOS Up)
void forward2() {
  for (int bit = 0; bit < MLOG; bit++)
    for (int i = MAXN - 1; i >= 0; i--)
      if (i & (1 << bit)) dp[i ^ (1 << bit)] += dp[i];
}
void backward2() {
  for (int bit = 0; bit < MLOG; bit++)
    for (int i = 0; i < MAXN; i++)
      if (i & (1 << bit)) dp[i ^ (1 << bit)] -= dp[i];
}
```

4 Game Theory

4.1 Points to be noted [14 lines] - f960ca72

>[First Write a Brute Force solution]
>Nim = all xor
>Misere Nim = Nim + corner case: if all piles are 1, reverse(nim)
>Bogus Nim = Nim
>Staircase Nim = Odd indexed pile Nim (Even indexed pile doesnt matter, as one player can give bogus moves to drop all even piles to ground)
>Sprague Grundy: [Every impartial game under the normal play convention is equivalent to a one-heap game of nim]
Every tree = one nim pile = tree root value; tree leaf value = 0; tree node value = mex of all child nodes.
[Careful: one tree node can become multiple new tree roots(multiple elements in one node), then the value of that node = xor of all those root values]
>Hackenbush(Given a rooted tree; cut an edge in one move; subtree under that edge gets removed; last player to cut wins):
Colon: //$G(u) = (G(v1)+1) \oplus (G(v2)+1) \oplus \cdots [v1, v2, \cdots$ are childs of u]
For multiple trees ans is their xor
>Hackenbush on graph (instead of tree given an rooted graph):

fusion: All edges in a cycle can be fused to get a tree structure; build a super node, connect some single nodes with that super node, number of single nodes is the number of edges in the cycle.
Sol: [Bridge component tree] mark all bridges, a group of edges that are not bridges, becomes one component and contributes number of edges to the hackenbush. (even number of edges contributes 0, odd number of edges contributes 1)

5 Geometry

5.1 Geometry Kit [213 lines] - 2cd03012

```
// T = ll for exact integer geometry, ld for real
typedef long double ld;
const ld EPS = 1e-9, PI = acosl(-1);
template <class T> struct P {
  T x, y;
  P(T x = 0, T y = 0) : x(x), y(y) {}
  P operator+(P o) const { return P(x + o.x, y + o.y);}
  P operator-(P o) const { return P(x - o.x, y - o.y);}
  P operator*(T k) const { return P(x * k, y * k); }
  P operator/(T k) const { return P(x / k, y / k); }
  bool operator<(P o) const {
    return x < o.x || (x == o.x && y < o.y);
  }
  bool operator==(P o) const {
    return x == o.x && y == o.y;
  }
  T dot(P o) const { return x * o.x + y * o.y; }
  T cross(P o) const { return x * o.y - y * o.x; }
  T cross(P a, P b) const { // (a - this) x (b - this)
    return (a - *this).cross(b - *this);
  }
  T norm2() const { return x * x + y * y; }
  ld len() const { return sqrtl((ld)norm2()); }
  ld ang() const { return atan2l((ld)y, (ld)x); }
  P perp() const { return P(-y, x); } // rotate +90
  P rot(ld a) const { // real T only
    return P(x * cosl(a) - y * sinl(a),
             x * sinl(a) + y * cosl(a));
  }
};
using pt = P<ll>; using ptd = P<ld>;
int sgn(ll v) { return (v > 0) - (v < 0); }
int sgn(ld v) { return (v > EPS) - (v < -EPS); }
// 1: c left of a->b (ccw), -1: right, 0: collinear
template <class T> int orient(P<T> a, P<T> b, P<T> c) {
  return sgn(a.cross(b, c));
}
template <class T> bool onSeg(P<T> p, P<T> a, P<T> b) {
  return orient(a, b, p) == 0 &&
         sgn((a - p).dot(b - p)) <= 0;
}
// segments ab and cd intersect (touching counts)
template <class T>
bool segInter(P<T> a, P<T> b, P<T> c, P<T> d) {
  if (onSeg(c, a, b) || onSeg(d, a, b) ||
      onSeg(a, c, d) || onSeg(b, c, d)) return true;
  return orient(a, b, c) * orient(a, b, d) < 0 &&
         orient(c, d, a) * orient(c, d, b) < 0;
}
// intersection of LINES ab, cd (must not be parallel)
ptd lineInter(ptd a, ptd b, ptd c, ptd d) {
  ld t = (c - a).cross(d - c) / (b - a).cross(d - c);
  return a + (b - a) * t;
}
ld distLine(ptd p, ptd a, ptd b) { // line ab
  return fabsl(a.cross(b, p)) / (b - a).len();
}
ld distSeg(ptd p, ptd a, ptd b) { // segment ab
  if (a == b) return (p - a).len();
  ptd d = b - a;
  ld t = (p - a).dot(d) / d.norm2();
  t = max((ld)0, min((ld)1, t));
  return (p - (a + d * t)).len();
}
ptd project(ptd p, ptd a, ptd b) { // foot on line ab
  ptd d = b - a;
  return a + d * ((p - a).dot(d) / d.norm2());
}
// ---- Polygons (vertices in order) ----
// 2 * signed area (> 0 if CCW). Pick: A = I + B/2 - 1
template <class T> T area2(const vector<P<T>>& p) {
  T s = 0;
  for (int i = 0, n = p.size(); i < n; i++)
    s += p[i].cross(p[(i + 1) % n]);
  return s;
}
// lattice points on segment ab (incl. both ends)
ll boundaryPts(pt a, pt b) {
  return __gcd(llabs(a.x - b.x), llabs(a.y - b.y)) + 1;
}
// 1 inside, 0 on boundary, -1 outside. O(n), any
    simple
template <class T>
int inPoly(const vector<P<T>>& p, P<T> q) {
  int n = p.size(); bool in = false;
  for (int i = 0; i < n; i++) {
    P<T> a = p[i], b = p[(i + 1) % n];
    if (onSeg(q, a, b)) return 0;
    if ((a.y > q.y) != (b.y > q.y) &&
        (b.y > a.y) == (sgn(a.cross(b, q)) > 0))
      in = !in;
  }
  return in ? 1 : -1;
}
// Convex hull, monotone chain, CCW, no collinear pts
// (change <= 0 to < 0 to keep collinear; not if all
// points are collinear)
template <class T> vector<P<T>> hull(vector<P<T>> p) {
  sort(p.begin(), p.end());
  p.erase(unique(p.begin(), p.end()), p.end());
  int n = p.size(), k = 0;
  if (n < 3) return p;
  vector<P<T>> h(2 * n);
  auto bad = [&](int i) { // not a strict left turn
    return sgn(h[k - 2].cross(h[k - 1], p[i])) <= 0;
  };
  for (int i = 0; i < n; h[k++] = p[i++])
    while (k >= 2 && bad(i)) k--;
  for (int i = n-2, t = k+1; i >= 0; h[k++] = p[i--])
    while (k >= t && bad(i)) k--;
  h.resize(k - 1);
  return h;
}
// q inside or on convex polygon h (CCW, from hull())
// O(log n)
template <class T>
bool inConvex(const vector<P<T>>& h, P<T> q) {
  int n = h.size();
  if (n == 1) return q == h[0];
  if (n == 2) return onSeg(q, h[0], h[1]);
  if (orient(h[0], h[1], q) < 0 ||
      orient(h[0], h[n - 1], q) > 0) return false;
  int l = 1, r = n - 1;
  while (r - l > 1) {
    int m = (l + r) / 2;
    if (orient(h[0], h[m], q) >= 0) l = m; else r = m;
  }
  return orient(h[l], h[r], q) >= 0;
}
// max squared distance between points of hull h
template <class T> T diameter2(const vector<P<T>>& h) {
  int n = h.size();
  if (n == 1) return 0;
  T best = 0;
  for (int i = 0, j = 1; i < n; i++) {
    int ni = (i + 1) % n;
    while ((h[ni] - h[i]).cross(h[(j + 1) % n] - h[j])
           > 0) j = (j + 1) % n;
    best = max({best, (h[i] - h[j]).norm2(),
                (h[ni] - h[j]).norm2()});
  }
  return best;
}
// closest pair of points, squared distance. O(n log n)
ll closestPair2(vector<pt> p) {
  sort(p.begin(), p.end());
  set<pair<ll, ll>> s; // (y, x) of points in the strip
  ll best = LLONG_MAX;
  for (int i = 0, j = 0; i < (int)p.size(); i++) {
    ll d = ceill(sqrtl((ld)best));
    while (j < i && p[i].x - p[j].x >= d)
      s.erase({p[j].y, p[j].x}), j++;
    auto it = s.lower_bound({p[i].y - d, LLONG_MIN});
    for (; it != s.end() && it->first <= p[i].y + d;
         ++it) {
      ll dx = p[i].x - it->second;
      ll dy = p[i].y - it->first;
      best = min(best, dx * dx + dy * dy);
    }
    s.insert({p[i].y, p[i].x});
  }
  return best;
}
// ---- Circles ----
ptd circumcenter(ptd a, ptd b, ptd c) {
  ptd B = b - a, C = c - a;
  ld d = 2 * B.cross(C);
  return a + ptd(C.y * B.norm2() - B.y * C.norm2(),
                 B.x * C.norm2() - C.x * B.norm2()) / d;
}
// minimum enclosing circle, expected O(n). {center, r}
pair<ptd, ld> mec(vector<ptd> p) {
  shuffle(p.begin(), p.end(), mt19937(1234));
  ptd o = p[0]; ld r = 0;
  auto out = [&](ptd q) {
    return (q - o).len() > r + EPS;
  };
  for (int i = 1; i < (int)p.size(); i++) {
    if (!out(p[i])) continue;
    o = p[i]; r = 0;
    for (int j = 0; j < i; j++) {
      if (!out(p[j])) continue;
      o = (p[i] + p[j]) / 2; r = (p[i] - o).len();
      for (int k = 0; k < j; k++) {
        if (!out(p[k])) continue;
```

```
        o = circumcenter(p[i], p[j], p[k]);
        r = (p[i] - o).len();
      }
    }
  }
  return {o, r};
}
// line ab with circle (o, r): 0, 1 or 2 points
vector<ptd> circleLine(ptd o, ld r, ptd a, ptd b) {
  ptd f = project(o, a, b);
  ld h2 = r * r - (f - o).norm2();
  if (h2 < -EPS) return {};
  ptd d = (b - a) / (b - a).len();
  d = d * sqrtl(max((ld)0, h2));
  if (h2 < EPS) return {f};
  return {f - d, f + d};
}
// circles (o1, r1), (o2, r2): intersection points
vector<ptd> circleCircle(ptd o1, ld r1, ptd o2, ld r2) {
  ptd d = o2 - o1; ld d2 = d.norm2();
  if (d2 < EPS) return {}; // concentric
  ld a = (d2 + r1 * r1 - r2 * r2) / (2 * d2);
  ld h2 = r1 * r1 - a * a * d2;
  if (h2 < -EPS) return {};
  ptd m = o1 + d * a;
  ptd off = d.perp() * sqrtl(max((ld)0, h2) / d2);
  if (h2 < EPS) return {m};
  return {m - off, m + off};
}
```

6 Graph

6.1 0-1 BFS [18 lines] - f5b209de

```
// Shortest path when weights are only 0 or 1, O(V + E)
// adj[u] = {v, w}, w in {0, 1}. Also grid problems.
vector<int> bfs01(int src,
                  vector<vector<pair<int, int>>>& adj) {
  vector<int> d(adj.size(), INT_MAX);
  deque<int> dq;
  d[src] = 0; dq.push_back(src);
  while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj[u])
      if (d[u] + w < d[v]) {
        d[v] = d[u] + w;
        if (w == 0) dq.push_front(v);
        else dq.push_back(v);
      }
  }
  return d;
}
```

6.2 2-SAT [48 lines] - 0a20c8b4

```
// n boolean vars 0..n-1. Literal: var x true = 2x,
// false = 2x+1. Tarjan SCC, O(n + clauses)
struct TwoSat {
  int n; vector<vector<int>> g;
  vector<int> comp, low, tin, stk; vector<bool> val;
  int timer = 0, nc = 0;
  TwoSat(int n) : n(n), g(2 * n) {}
  int lit(int x, bool v) { return 2 * x + !v; }
  // (x == vx) OR (y == vy)
  void either(int x, bool vx, int y, bool vy) {
    int a = lit(x, vx), b = lit(y, vy);
    g[a ^ 1].push_back(b); g[b ^ 1].push_back(a);
  }
  void setVal(int x, bool v) { either(x, v, x, v); }
  // (x == vx) -> (y == vy)
  void implies(int x, bool vx, int y, bool vy) {
    either(x, !vx, y, vy);
  }
  void dfs(int u) {
    low[u] = tin[u] = ++timer; stk.push_back(u);
    for (int v : g[u]) {
      if (!tin[v]) {
        dfs(v); low[u] = min(low[u], low[v]);
      } else if (comp[v] == -1) {
        low[u] = min(low[u], tin[v]);
      }
    }
    if (low[u] == tin[u]) {
      int v;
      do {
        v = stk.back(); stk.pop_back(); comp[v] = nc;
      } while (v != u);
      nc++;
    }
  }
  bool solve() { // val[x] = assignment if satisfiable
    comp.assign(2 * n, -1); low.assign(2 * n, 0);
    tin.assign(2 * n, 0); val.assign(n, false);
    for (int i = 0; i < 2 * n; i++)
      if (!tin[i]) dfs(i);
    for (int x = 0; x < n; x++) {
      if (comp[2 * x] == comp[2 * x + 1]) return false;
      // Tarjan: SCC ids are in reverse topo order
      val[x] = comp[2 * x] < comp[2 * x + 1];
    }
    return true;
  }
};
```

6.3 Articulation Bridges [40 lines] - f1f88788

```
const int N = 2e5 + 5;
vector < pair < int, int >> g[N];
int vis[N], st[N], low[N];
int clk, n;
vector < pair < int, int >> bridges;
void dfs(int u, int p, int edg) {
  vis[u] = 1;
  ++clk;
  st[u] = clk;
  low[u] = clk;
  for (auto[v, id]: g[u]) {
    // handled parallel edges between u, v
    if (v == p && id == edg) continue;
    if (vis[v]) {
      low[u] = min(low[u], st[v]);
    } else {
      dfs(v, u, id);
      low[u] = min(low[u], low[v]);
      if (st[u] < low[v]) {
        // edge u, v has bridge
        bridges.push_back({
          u,
          v
        });
      }
    }
  }
}
void find_bridges() {
  clk = 0;
  for (int i = 1; i <= n; i++) {
    vis[i] = 0;
    st[i] = low[i] = -1;
  }
  for (int i = 1; i <= n; i++) {
    if (!vis[i]) {
      dfs(i, 0, -1);
    }
  }
}
```

6.4 Articulation Points [39 lines] - 50d23cbe

```
const int N = 2e5 + 5;
vector < int > g[N];
int vis[N], st[N], low[N], adjComp[N];
int clk, n;
void dfs(int u, int p) {
  vis[u] = 1;
  ++clk;
  st[u] = clk;
  low[u] = clk;
  int child = 0;
  for (int v: g[u]) {
    if (v == p) continue;
    if (vis[v]) {
      low[u] = min(low[u], st[v]);
    } else {
      dfs(v, u);
      low[u] = min(low[u], low[v]);
      if (low[v] >= st[u] && p != 0) {
        adjComp[u]++;
      }
      child++;
    }
  }
  if (p == 0 && child > 1) {
    adjComp[u] = child - 1;
  }
}
void find_vertices() {
  clk = 0;
  for (int i = 1; i <= n; i++) {
    vis[i] = 0;
    st[i] = low[i] = -1;
  }
  for (int i = 1; i <= n; i++) {
    if (!vis[i]) {
      dfs(i, 0);
    }
  }
}
```

6.5 Bellmanford [40 lines] - 928d2159

```
// Nodes 1..n. Both return false if a NEGATIVE CYCLE is
// reachable from src. d[v] = INF if unreachable.
const ll INF = 1e18;
struct Edge { int u, v; ll w; };
// Bellman-Ford O(V * E)
bool bellmanFord(int n, int src, vector<Edge>& e,
                 vector<ll>& d) {
  d.assign(n + 1, INF); d[src] = 0;
  for (int it = 1; it <= n; it++) {
    bool any = false;
    for (auto [u, v, w] : e)
      if (d[u] < INF && d[u] + w < d[v]) {
        d[v] = d[u] + w; any = true;
```

```
      if (it == n) return false; // n-th pass relaxed
    }
    if (!any) break;
  }
  return true;
}
// SPFA: usually fast, worst O(V * E). adj[u] = {v, w}
bool spfa(int n, int src,
          vector<vector<pair<int, ll>>>& adj,
          vector<ll>& d) {
  d.assign(n + 1, INF);
  vector<int> len(n + 1, 0); // edges on current path
  vector<bool> inq(n + 1, false);
  queue<int> q;
  d[src] = 0; q.push(src); inq[src] = true;
  while (!q.empty()) {
    int u = q.front(); q.pop(); inq[u] = false;
    for (auto [v, w] : adj[u])
      if (d[u] + w < d[v]) {
        d[v] = d[u] + w;
        len[v] = len[u] + 1;
        if (len[v] >= n) return false;
        if (!inq[v]) q.push(v), inq[v] = true;
      }
  }
  return true;
}
```

6.6 Block Cut Tree [76 lines] - a64cd41b

```
const int N = 2e5 + 5;
vector<int> g[N], bct[2 * N];
int vis[N], low[N], disc[N], is_art[N];
stack<int> stk;
vector<int> block_of[N];
int clk, n, m, bct_nodes, root;
void dfs(int u, int p) {
  vis[u] = 1;
  disc[u] = low[u] = ++clk;
  stk.push(u);
  int child = 0;
  for (int v : g[u]) {
    if (v == p) continue;
    if (!vis[v]) {
      child++;
      dfs(v, u);
      low[u] = min(low[u], low[v]);
      if ((p == -1 && child > 1) ||
          (p != -1 && low[v] >= disc[u])) {
        is_art[u] = 1;
        vector<int> block;
        while (stk.top() != v) {
          block.push_back(stk.top());
          stk.pop();
        }
        block.push_back(stk.top());
        stk.pop();
        block.push_back(u);
        int block_id = ++bct_nodes;
        for (int x : block) {
          block_of[x].push_back(block_id);
          if (is_art[x]) {
            bct[x].push_back(block_id);
            bct[block_id].push_back(x);
          }
        }
      }
    } else {
      low[u] = min(low[u], disc[v]);
    }
  }
}
void build_bct() {
  clk = 0;
  bct_nodes = n;
  for (int i = 1; i <= n; i++) {
    vis[i] = is_art[i] = 0;
    block_of[i].clear();
    bct[i].clear();
  }
  for (int i = n + 1; i <= 2 * n; i++) bct[i].clear();
  for (int i = 1; i <= n; i++) {
    if (!vis[i]) {
      dfs(i, -1);
      if (!stk.empty()) {
        int block_id = ++bct_nodes;
        while (!stk.empty()) {
          int x = stk.top();
          stk.pop();
          block_of[x].push_back(block_id);
          if (is_art[x]) {
            bct[x].push_back(block_id);
            bct[block_id].push_back(x);
          }
        }
      }
    }
  }
  root = n + 1;
  for (int i = 1; i <= n; i++) {
    if (is_art[i]) {
      root = i;
      break;
    }
  }
}
```

6.7 Centroid Decomposition [30 lines] - 2fb8ca4f

```
const int N = 1e5 + 5;
vector<int> g[N];
int sz[N], dead[N], cenpar[N];
void dfs_sz(int u, int p) {
  sz[u] = 1;
  for (int v : g[u]) {
    if (v == p || dead[v]) continue;
    dfs_sz(v, u);
    sz[u] += sz[v];
  }
}
int find_centroid(int num, int u, int p) {
  for (int v : g[u]) {
    if (v != p && !dead[v] && 2 * sz[v] > num) {
      return find_centroid(num, v, u);
    }
  }
  return u;
}
void decompose(int u, int p) {
  dfs_sz(u, p);
  int cent = find_centroid(sz[u], u, p);
  dead[cent] = true;
  cenpar[cent] = p;
  for (int v : g[cent]) {
    if (!dead[v]) {
      decompose(v, cent);
    }
  }
}
```

6.8 DSU on Tree (Small to Large) [38 lines] - ed577778

```
// Answer a query for EVERY subtree, O(n log n) (sack)
// Example: number of distinct colors in subtree of u.
// Edit add()/remove effect and the answer line only.
const int N = 2e5 + 5;
vector<int> g[N];
int col[N], sz[N], big[N], tin[N], tout[N], ver[N], T;
int cnt[N], distinct_, ans[N];
void dfs_sz(int u, int p) {
  sz[u] = 1; big[u] = -1;
  tin[u] = T; ver[T++] = u;
  for (int v : g[u]) if (v != p) {
    dfs_sz(v, u); sz[u] += sz[v];
    if (big[u] == -1 || sz[v] > sz[big[u]]) big[u] = v;
  }
  tout[u] = T; // subtree = ver[tin[u] .. tout[u]-1]
}
void add(int u, int d) { // d = +1 add u, -1 remove u
  if (d == 1 && cnt[col[u]]++ == 0) distinct_++;
  if (d == -1 && --cnt[col[u]] == 0) distinct_--;
}
void dfs(int u, int p, bool keep) {
  for (int v : g[u])
    if (v != p && v != big[u]) dfs(v, u, false);
  if (big[u] != -1) dfs(big[u], u, true); // keep heavy
  for (int v : g[u])
    if (v != p && v != big[u])
      for (int i = tin[v]; i < tout[v]; i++)
        add(ver[i], 1);
  add(u, 1);
  ans[u] = distinct_; // answer queries of u here
  if (!keep)
    for (int i = tin[u]; i < tout[u]; i++)
      add(ver[i], -1);
}
// dfs_sz(1, 0); dfs(1, 0, true);
// Small-to-large merging of sets: keep the bigger
// if (S[a].size() < S[b].size()) swap(S[a], S[b]);
// for (auto x : S[b]) S[a].insert(x); -> O(n log^2 n)
```

6.9 Dijkstra [20 lines] - 1fbb9433

```
// adj[u] = {v, w}, w >= 0. dist = INF if unreachable
const ll INF = 1e18;
vector<ll> dijkstra(int src,
                    vector<vector<pair<int, ll>>>& adj) {
  int n = adj.size();
  vector<ll> d(n, INF);
  priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                 greater<pair<ll, int>>> pq;
  d[src] = 0; pq.push({0, src});
  while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if (du != d[u]) continue; // stale entry
    for (auto [v, w] : adj[u])
      if (du + w < d[v]) {
        d[v] = du + w;
        pq.push({d[v], v});
      }
  }
  return d;
}
```

6.10 Dinic Max Flow [49 lines] - 974cda68

```cpp
// O(V^2 E), unit capacity / bipartite: O(E sqrt V)
// Min cut: after maxflow, S side = level[u] != -1
// Bipartite matching: s->L, L->R, R->t, all cap 1
struct Dinic {
  struct E { int to; ll cap; };
  vector<E> e; vector<vector<int>> g;
  vector<int> level, it;
  Dinic(int n) : g(n), level(n), it(n) {}
  int addEdge(int u, int v, ll cap, ll rcap = 0) {
    g[u].push_back(e.size()); e.push_back({v, cap});
    g[v].push_back(e.size()); e.push_back({u, rcap});
    return e.size() - 2; // flow on id = e[id ^ 1].cap
  }
  bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    queue<int> q; q.push(s); level[s] = 0;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int id : g[u])
        if (e[id].cap > 0 && level[e[id].to] == -1) {
          level[e[id].to] = level[u] + 1;
          q.push(e[id].to);
        }
    }
    return level[t] != -1;
  }
  ll dfs(int u, int t, ll f) {
    if (u == t) return f;
    for (int& i = it[u]; i < (int)g[u].size(); i++) {
      int id = g[u][i], v = e[id].to;
      if (e[id].cap > 0 && level[v] == level[u] + 1) {
        ll got = dfs(v, t, min(f, e[id].cap));
        if (got > 0) {
          e[id].cap -= got; e[id ^ 1].cap += got;
          return got;
        }
      }
    }
    return 0;
  }
  ll maxflow(int s, int t) {
    ll flow = 0;
    while (bfs(s, t)) {
      fill(it.begin(), it.end(), 0);
      while (ll f = dfs(s, t, LLONG_MAX)) flow += f;
    }
    return flow;
  }
};
```

6.11 Euler Path (Hierholzer) [30 lines] - 65d66715

```cpp
// Euler path/circuit using every edge once. O(V + E)
// Undirected: 0 or 2 odd-degree vertices (start at
    odd)
// Directed: out-in = 1 at start, in-out = 1 at end,
// else 0. Returns vertices; size != E + 1 ->
    impossible
// (also when edges are in more than one component)
vector<int> eulerPath(int n,
    vector<pair<int, int>>& edges,
    bool directed, int start) {
  vector<vector<pair<int, int>>> g(n); // {to, edge id}
  for (int i = 0; i < (int)edges.size(); i++) {
    auto [u, v] = edges[i];
    g[u].push_back({v, i});
    if (!directed) g[v].push_back({u, i});
  }
  vector<bool> used(edges.size(), false);
  vector<int> ptr(n, 0), path, stk = {start};
  while (!stk.empty()) {
    int u = stk.back();
    while (ptr[u] < (int)g[u].size() &&
           used[g[u][ptr[u]].second]) ptr[u]++;
    if (ptr[u] == (int)g[u].size()) {
      path.push_back(u); stk.pop_back();
    } else {
      auto [v, id] = g[u][ptr[u]++];
      used[id] = true; stk.push_back(v);
    }
  }
  reverse(path.begin(), path.end());
  return path;
}
```

6.12 Euler Tour (Tree Flatten) [13 lines] - 7bf8c312

```cpp
// Subtree of u = flat[tin[u] .. tout[u]] (contiguous)
// -> subtree query = range query (BIT / segtree)
const int N = 2e5 + 5;
vector<int> g[N];
int tin[N], tout[N], flat[N], timer_ = 0;
void dfs(int u, int p) {
  tin[u] = timer_;
  flat[timer_++] = u;
  for (int v : g[u])
    if (v != p) dfs(v, u);
  tout[u] = timer_ - 1;
}
// v in subtree of u <=> tin[u] <= tin[v] <= tout[u]
```

6.13 Floyed Warshall [13 lines] - 32069763

```cpp
// Init: dist[i][i] = 0, dist[u][v] = min(w), else INF
// Negative cycle exists iff dist[i][i] < 0 after run
const ll INF = 1e18;
const int MAXN = 505;
ll dist[MAXN][MAXN];
void floydWarshall(int n) {
  for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
      for (int j = 0; j < n; ++j)
        if (dist[i][k] < INF && dist[k][j] < INF)
          dist[i][j] =
              min(dist[i][j], dist[i][k] + dist[k][j]);
}
```

6.14 Graph Basics [55 lines] - 251fa6c0

```cpp
// 4-dir, 8-dir (first 4 + diagonals), knight moves
int dx[] = {1, -1, 0, 0, 1, 1, -1, -1};
int dy[] = {0, 0, 1, -1, 1, -1, 1, -1};
int kx[] = {2, 1, -1, -2, -2, -1, 1, 2};
int ky[] = {1, 2, 2, 1, -1, -2, -2, -1};
// Grid BFS from (sx, sy), '#' = wall, -1 = unreachable
vector<vector<int>> gridBFS(vector<string>& g, int sx,
                            int sy) {
  int n = g.size(), m = g[0].size();
  vector<vector<int>> d(n, vector<int>(m, -1));
  queue<pair<int, int>> q;
  d[sx][sy] = 0; q.push({sx, sy});
  while (!q.empty()) {
    auto [x, y] = q.front(); q.pop();
    for (int k = 0; k < 4; k++) {
      int nx = x + dx[k], ny = y + dy[k];
      if (nx < 0 || ny < 0 || nx >= n || ny >= m)
        continue;
      if (g[nx][ny] == '#' || d[nx][ny] != -1)
        continue;
      d[nx][ny] = d[x][y] + 1; q.push({nx, ny});
    }
  }
  return d;
}
// Bipartite check (2-coloring), col[v] in {0, 1}
bool bipartite(vector<vector<int>>& adj,
               vector<int>& col) {
  int n = adj.size(); col.assign(n, -1);
  for (int s = 0; s < n; s++) {
    if (col[s] != -1) continue;
    queue<int> q; q.push(s); col[s] = 0;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : adj[u]) {
        if (col[v] == -1)
          col[v] = col[u] ^ 1, q.push(v);
        else if (col[v] == col[u]) return false;
      }
    }
  }
  return true;
}
// Directed cycle: 0 unvisited, 1 on stack, 2 done
bool hasCycle(int u, vector<vector<int>>& adj,
              vector<int>& st) {
  st[u] = 1;
  for (int v : adj[u]) {
    if (st[v] == 1) return true;
    if (st[v] == 0 && hasCycle(v, adj, st))
      return true;
  }
  st[u] = 2;
  return false;
}
```

6.15 Heavy Light Decomposition [80 lines] - 4efd2fd3

```cpp
// g[u] = {v, edge_id}; edge ids 1..n-1
vector<pair<int, int>> g[N];
int edge_to_ch[N];
int par[N], dep[N], sz[N];
void dfs(int u, int p = 0) {
  sz[u] = 1, par[u] = p;
  dep[u] = dep[p] + 1;
  int mx = 0;
  for (auto &[v, idx] : g[u]) {
    if (v == p) continue;
    edge_to_ch[idx] = v;
    dfs(v, u);
    sz[u] += sz[v];
    if (sz[v] > mx) {
      mx = sz[v];
      swap(v, g[u][0].first);
    }
  }
}
int T, head[N], st[N], en[N];
void dfs_hld(int u, int p = 0) {
  st[u] = ++T;
  head[u] = (p != 0 && g[p][0].first == u) ? head[p]
                                          : u;
  for (auto [v, idx] : g[u]) {
    if (v == p) continue;
```

```
    dfs_hld(v, u);
  }
  en[u] = T;
}
int lca(int a, int b) {
  for (; head[a] != head[b]; b = par[head[b]]) {
    if (dep[head[a]] > dep[head[b]]) swap(a, b);
  }
  if (dep[a] > dep[b]) swap(a, b);
  return a;
}
int n;
// process node u upto it's ancestor a
// if excl is true, a won't process
ll chain_process(int a, int u, bool excl = false) {
  ll ans = 0;
  for (; head[u] != head[a]; u = par[head[u]]) {
    ans = max(ans, query(1, 1, n, st[head[u]], st[u]));
  }
  ans = max(ans, query(1, 1, n, st[a] + excl, st[u]));
  return ans;
}
// process path from node a to b
// if excl is true, lca won't process
ll path_process(int a, int b, bool excl = false) {
  ll ans = 0;
  for (; head[a] != head[b]; b = par[head[b]]) {
    if (dep[head[a]] > dep[head[b]]) swap(a, b);
    ans = max(ans, query(1, 1, n, st[head[b]], st[b]));
  }
  if (dep[a] > dep[b]) swap(a, b);
  ans = max(ans, query(1, 1, n, st[a] + excl, st[b]));
  return ans;
}
// update path from node u to v
// if excl is true, lca won't update
void path_update(int a, int b, int val,
                 bool excl = false) {
  for (; head[a] != head[b]; b = par[head[b]]) {
    if (dep[head[a]] > dep[head[b]]) swap(a, b);
    update(1, 1, n, st[head[b]], st[b], val);
  }
  if (dep[a] > dep[b]) swap(a, b);
  update(1, 1, n, st[a] + excl, st[b], val);
}
/* Needs a segtree with query(1,1,n,l,r) (max here) and
   update(1,1,n,l,r,val). Usage:
   dfs(1); head[1] = 1; dfs_hld(1);
   node value: arr[st[u]] = val[u]
   edge value: store on child, arr[st[edge_to_ch[i]]]
     = cost[i], then use excl = true (skip lca)
   build(1, 1, n);
   subtree of u = [st[u], en[u]]
*/
```

6.16 Hopcroft-Karp Matching [49 lines] - 01d1a1be

```
// Max bipartite matching, O(E sqrt V). Left 0..n-1,
// right 0..m-1, adj[u] = right neighbours of left u.
// Konig: min vertex cover = max matching,
// max independent set = n + m - matching
struct HopcroftKarp {
  int n, m;
  vector<vector<int>> adj;
  vector<int> mL, mR, dist;
  HopcroftKarp(int n, int m)
      : n(n), m(m), adj(n), mL(n, -1), mR(m, -1),
        dist(n) {}
  void addEdge(int u, int v) { adj[u].push_back(v); }
  bool bfs() {
    queue<int> q; bool found = false;
    for (int u = 0; u < n; u++) {
      dist[u] = mL[u] == -1 ? 0 : -1;
      if (!dist[u]) q.push(u);
    }
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : adj[u]) {
        int w = mR[v];
        if (w == -1) found = true;
        else if (dist[w] == -1)
          dist[w] = dist[u] + 1, q.push(w);
      }
    }
    return found;
  }
  bool dfs(int u) {
    for (int v : adj[u]) {
      int w = mR[v];
      if (w == -1 ||
          (dist[w] == dist[u] + 1 && dfs(w))) {
        mL[u] = v; mR[v] = u;
        return true;
      }
    }
    dist[u] = -1;
    return false;
  }
  int maxMatching() {
    int res = 0;
    while (bfs())
      for (int u = 0; u < n; u++)
        if (mL[u] == -1 && dfs(u)) res++;
    return res;
  }
};
```

6.17 K'th Shortest path [19 lines] - 5689f9fe

```
// d[u] = K smallest walk lengths src -> u (sorted;
// walks may repeat vertices). Node popped <= K times.
// O(K * (V + E) log). adj[u] = {v, w}, w >= 0
vector<vector<ll>> kShortest(int src, int K,
    vector<vector<pair<int, ll>>>& adj) {
  int n = adj.size();
  vector<vector<ll>> d(n);
  priority_queue<pair<ll, int>, vector<pair<ll, int>>,
                 greater<pair<ll, int>>> pq;
  pq.push({0, src});
  while (!pq.empty()) {
    auto [du, u] = pq.top(); pq.pop();
    if ((int)d[u].size() >= K) continue;
    d[u].push_back(du);
    for (auto [v, w] : adj[u])
      if ((int)d[v].size() < K) pq.push({du + w, v});
  }
  return d;
}
```

6.18 Kruskal [14 lines] - a44dd888

```
// MST using DSU (Data Structure section). Nodes 1..n
// edges: {w, u, v}. Returns MST weight, fills `used`.
ll kruskal(int n, vector<array<ll, 3>>& edges,
           vector<array<ll, 3>>& used) {
  sort(edges.begin(), edges.end());
  DSU d(n); ll total = 0;
  for (auto& [w, u, v] : edges)
    if (d.find(u) != d.find(v)) {
      d.unite(u, v);
      total += w;
      used.push_back({w, u, v});
    }
  return total; // if used.size() < n - 1: not
      connected
}
```

6.19 LCA [43 lines] - 917fd387

```
// 1-indexed, root's parent = 0. dfs(root, 0) first
const int N = 2e5 + 5, LG = 20; // 2^LG > N
vector<int> g[N];
int depth[N], parent[N][LG];
void dfs(int u, int p) {
  depth[u] = depth[p] + 1;
  parent[u][0] = p;
  for (int i = 1; i < LG; i++) {
    parent[u][i] = parent[parent[u][i - 1]][i - 1];
  }
  for (int v : g[u]) {
    if (v == p) continue;
    dfs(v, u);
  }
}
int lca(int u, int v) {
  if (depth[u] < depth[v]) swap(u, v);
  int diff = depth[u] - depth[v];
  for (int i = 0; i < LG; i++) {
    if (diff >> i & 1) {
      u = parent[u][i];
    }
  }
  if (u == v) return u;
  for (int i = LG - 1; i >= 0; i--) {
    if (parent[u][i] != parent[v][i]) {
      u = parent[u][i];
      v = parent[v][i];
    }
  }
  return parent[u][0];
}
int dist(int u, int v) {
  return depth[u] + depth[v] - 2 * depth[lca(u, v)];
}
int kthParent(int u, int k) { // 0 if k > depth
  for (int i = 0; i < LG; i++) {
    if (k >> i & 1) {
      u = parent[u][i];
    }
  }
  return u;
}
```

6.20 Min Cost Max Flow [42 lines] - 82bfb9f8

```
// Successive shortest paths + SPFA (negative costs ok,
// no negative cycles). Returns {max flow, min cost}.
// Flow limit K: pass maxf = K. Max cost: negate costs.
struct MCMF {
  struct E { int to; ll cap, cost; };
  int n; vector<E> e; vector<vector<int>> g;
  vector<ll> dist; vector<int> pe; vector<bool> inq;
  MCMF(int n) : n(n), g(n), dist(n), pe(n), inq(n) {}
  void addEdge(int u, int v, ll cap, ll cost) {
```

```
    g[u].push_back(e.size());
    e.push_back({v, cap, cost});
    g[v].push_back(e.size());
    e.push_back({u, 0, -cost});
  }
  bool spfa(int s, int t) {
    fill(dist.begin(), dist.end(), LLONG_MAX);
    queue<int> q; q.push(s); dist[s] = 0;
    while (!q.empty()) {
      int u = q.front(); q.pop(); inq[u] = false;
      for (int id : g[u]) {
        auto& [v, cap, cost] = e[id];
        if (cap > 0 && dist[u] + cost < dist[v]) {
          dist[v] = dist[u] + cost; pe[v] = id;
          if (!inq[v]) inq[v] = true, q.push(v);
        }
      }
    }
    return dist[t] != LLONG_MAX;
  }
  pair<ll, ll> run(int s, int t, ll maxf = LLONG_MAX) {
    ll flow = 0, cost = 0;
    while (flow < maxf && spfa(s, t)) {
      ll f = maxf - flow;
      for (int v = t; v != s; v = e[pe[v] ^ 1].to)
        f = min(f, e[pe[v]].cap);
      for (int v = t; v != s; v = e[pe[v] ^ 1].to)
        e[pe[v]].cap -= f, e[pe[v] ^ 1].cap += f;
      flow += f; cost += f * dist[t];
    }
    return {flow, cost};
  }
};
```

6.21 SCC [49 lines] - e4c242f2

```
vector<bool> visited;
// dfs from v; vertex appended to `output` on exit
void dfs(int v, vector<vector<int>> const &adj,
         vector<int> &output) {
  visited[v] = true;
  for (auto u : adj[v])
    if (!visited[u])
      dfs(u, adj, output);
  output.push_back(v);
}
// input: adj -- adjacency list of G
// output: comps -- strongly connected components of G
// output: adj_cond -- adj list of G^SCC (by roots)
void scc(vector<vector<int>> const &adj,
         vector<vector<int>> &comps,
         vector<vector<int>> &adj_cond) {
  int n = adj.size();
  comps.clear(), adj_cond.clear();
  vector<int> order; // vertices sorted by exit time
  visited.assign(n, false);
  // first series of depth first searches
  for (int i = 0; i < n; i++)
    if (!visited[i])
      dfs(i, adj, order);
  // create adjacency list of G^T
  vector<vector<int>> adj_rev(n);
  for (int v = 0; v < n; v++)
    for (int u : adj[v])
      adj_rev[u].push_back(v);
  visited.assign(n, false);
  reverse(order.begin(), order.end());
  vector<int> roots(n, 0); // root vertex of v's SCC
  // second series of depth first searches
  for (auto v : order)
    if (!visited[v]) {
      std::vector<int> comp;
      dfs(v, adj_rev, comp);
      comps.push_back(comp);
      int root = *min_element(begin(comp), end(comp));
      for (auto u : comp)
        roots[u] = root;
    }
  // add edges to condensation graph
  adj_cond.assign(n, {});
  for (int v = 0; v < n; v++)
    for (auto u : adj[v])
      if (roots[v] != roots[u])
        adj_cond[roots[v]].push_back(roots[u]);
}
```

6.22 Topological Sort [32 lines] - f160c26e

```
vector<int> topoSort(int V, vector<vector<int>>& g) {
  vector<int> inDegree(V, 0);
  vector<int> result;
  queue<int> q;
  for (int u = 0; u < V; u++) {
    for (int v : g[u]) {
      inDegree[v]++;
    }
  }
  for (int i = 0; i < V; i++) {
    if (inDegree[i] == 0) {
      q.push(i);
    }
  }
  while (!q.empty()) {
    int u = q.front();
    q.pop();
    result.push_back(u);
    for (int v : g[u]) {
      inDegree[v]--;
      if (inDegree[v] == 0) {
        q.push(v);
      }
    }
  }
  // May need to check inDegree of all vertices is 0
  if (result.size() != V) {
    cout << "Graph has a cycle, no topo sort!" << endl;
    return {};
  }
  return result;
}
```

6.23 Tree Rerooting [94 lines] - 2f321ddc

```
// ei = edge index (position of neighbour in g[u])
// merge    : Agg * Value * Vertex(int) * ei -> Agg
// base     : Vertex(int) -> Agg
// finalize : Agg * Vertex(int) * ei -> Value
namespace reroot {
const auto exclusive = [](const auto& a,
    const auto& base, const auto& merge, int vertex) {
  int n = (int)a.size();
  using Agg = decay_t<decltype(base)>;
  vector<Agg> b(n, base);
  for (int bit = (int)__lg(n); bit >= 0; --bit) {
    for (int i = n - 1; i >= 0; --i) b[i] = b[i >> 1];
    int sz = n - (n & !bit);
    for (int i = 0; i < sz; ++i) {
      int index = (i >> bit) ^ 1;
      b[index] = merge(b[index], a[i], vertex, i);
    }
  }
  return b;
};
const auto rerooter = [](const auto& g,
    const auto& base, const auto& merge,
    const auto& finalize) {
  int n = (int)g.size();
  using Agg = decay_t<decltype(base(0))>;
  using Value =
      decay_t<decltype(finalize(base(0), 0, 0))>;
  vector<Value> root_dp(n), dp(n);
  vector<vector<Value>> edge_dp(n), redge_dp(n);
  vector<int> bfs, parent(n);
  bfs.reserve(n);
  bfs.push_back(0);
  for (int i = 0; i < n; ++i) {
    int u = bfs[i];
    for (auto v : g[u]) {
      if (parent[u] == v) continue;
      parent[v] = u;
      bfs.push_back(v);
    }
  }
  for (int i = n - 1; i >= 0; --i) {
    int u = bfs[i];
    int par_ei = -1;
    Agg agg = base(u);
    for (int ei = 0; ei < (int)g[u].size(); ++ei) {
      int v = g[u][ei];
      if (parent[u] == v) {
        par_ei = ei;
        continue;
      }
      agg = merge(agg, dp[v], u, ei);
    }
    dp[u] = finalize(agg, u, par_ei);
  }
  for (auto u : bfs) {
    dp[parent[u]] = dp[u];
    edge_dp[u].reserve(g[u].size());
    for (auto v : g[u]) edge_dp[u].push_back(dp[v]);
    auto dp_excl =
        exclusive(edge_dp[u], base(u), merge, u);
    redge_dp[u].reserve(g[u].size());
    for (int i = 0; i < (int)dp_excl.size(); ++i) {
      auto merge_value = finalize(dp_excl[i], u, i);
      redge_dp[u].push_back(merge_value);
    }
    root_dp[u] = finalize(n > 1 ? merge(dp_excl[0],
        edge_dp[u][0], u, 0) : base(u), u, -1);
    for (int i = 0; i < (int)g[u].size(); ++i) {
      dp[g[u][i]] = redge_dp[u][i];
    }
  }
  return make_tuple(move(root_dp), move(edge_dp),
                    move(redge_dp));
};
}  // namespace reroot
void solve() {
  using Agg = int;
  using Value = int;
```

```
  auto base = [](int vertex) -> Agg { return 0; };
  auto merge = [](Agg vertex_dp, Value neighbor_dp,
                  int vertex, int ei) -> Agg {
    return vertex_dp + max(neighbor_dp, 0);
  };
  auto finalize = [&](Agg vertex_dp, int vertex,
                      int ei) -> Value {
    return vertex_dp + 2 * color[vertex] - 1;
  };
  auto [reroot_result, edge_dp, redge_dp] =
      reroot::rerooter(g, base, merge, finalize);
  for (auto dp : reroot_result) {
    cout << dp << ' ';
  }
  cout << '\n';
}
```

7 Misc

7.1 Bit hacks [16 lines] - df69591c

```
# x & -x is the least bit in x.
# iterate over all the subsets of the mask
for (int s=m; ; s=(s-1)&m) {
 ... you can use s ...
 if (s==0)  break;
}
# c = x&-x, r = x+c; (((r^x) >> 2) / c) | r is the
next number after x with the same number of bits set.
# __builtin_popcount(x) //number of ones in binary
  __builtin_popcountll(x) // for long long
# __builtin_clz(x) // number of leading zeros
  __builtin_ctz(x) // number of trailing zeros, they
      also have long long version
# bitset<N> b; //N is size of bitset
  b.count(); // return number of bits set O(N / 64)
  b.to_string(); // return string representation of
      bitset
  b.any() // return true if any bit is set O(N / 64)
```

7.2 Bitset C++ [14 lines] - e54c44bb

```
bitset<17>BS;
BS[1] = BS[7] = 1;
cout<<BS._Find_first()<<endl; // prints 1
bs._Find_next(idx): first set bit after idx. e.g.:
bitset<17>BS;
BS[1] = BS[7] = 1;
// prints 7,7
cout<<BS._Find_next(1)<<','<<BS._Find_next(3)<<endl;
So this code will print all of the set bits of BS:
for(int i = BS._Find_first(); i < BS.size();
    i = BS._Find_next(i))
    cout<<i<<endl;
// If no set bit after idx, BS._Find_next(idx) returns
// BS.size(); same for BS._Find_first() if BS is clear
```

7.3 Random Generator [7 lines] - c0453194

```
#define accuracy \

      chrono::steady_clock::now().time_since_epoch().count()
mt19937 rng(accuracy);
int rand(int l, int r) {
  uniform_int_distribution<int> ludo(l, r);
  return ludo(rng);
}
```

7.4 Special DS [25 lines] - 0c66ecd2

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T> using o_set = tree<T, null_type,
  less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;
template <typename T> using o_multiset =
  tree<pair<T, int>, null_type, less<pair<T, int>>,
  rb_tree_tag, tree_order_statistics_node_update>;
/*
o_set<int> st;
*st.find_by_order(0); // 0-th element
o_multiset<int> mst;
int timer = 0; // global/local timer for unique IDs
mst.insert({5, timer++});
// Erases exactly ONE instance of 5
mst.erase(mst.lower_bound({5, 0}));
mst.order_of_key({5, 0}); // strictly less than 5
*/
// Faster hash
gp_hash_table<int, int> mp;
// Or
unordered_map<int, int> mp;
mp.reserve(n * 2);
mp.max_load_factor(0.7);
```

7.5 Template [22 lines] - fb722b4f

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
#define all(x) (x).begin(), (x).end()
#ifdef ONPC
#define debug(x) cerr << #x << " = " << (x) << endl
#else
#define debug(x)
#endif
void solve() {
}
int main() {
  ios_base::sync_with_stdio(false); cin.tie(nullptr);
  int tc = 1;
  cin >> tc;
  for (int t = 1; t <= tc; t++) {
    // cout << "Case " << t << ": ";
    solve();
  }
}
```

7.6 Usefull Things [9 lines] - c67addfa

```
C++:
#pragma GCC optimize("Ofast")
#pragma GCC target("avx,avx2,fma")
ios_base::sync_with_stdio(false), cin.tie(nullptr),
    cout.tie(nullptr);
freopen("input.txt", "r", stdin);
Python:
import sys
input = sys.stdin.readline
sys.stdout.write("------")
```

7.7 VScode setup [7 lines] - 3e225a41

```
{
   "key": "f5",
   "command": "workbench.action.terminal.sendSequence",
   "args": {
      "text": "clear && g++
          ${fileBasenameNoExtension}.cpp -o
          ${fileBasenameNoExtension} &&
          ./${fileBasenameNoExtension} < in.txt >
          out.txt\n"
   }
} // need "files.autoSaveDelay": 100
```

7.8 stress [25 lines] - 4a84e7fd

```
#!/usr/bin/env bash
g++ -g "$1".cpp -DONPC -o "$1"
g++ -g "$2".cpp -DONPC -o "$2"
g++ -g "$3".cpp -DONPC -o "$3"
for ((testNum=0;testNum<$4;testNum++))
do
  ./$3 > stdinput
  ./$2 < stdinput > outSlow
  ./$1 < stdinput > outWrong
  H1=`md5sum outWrong`
  H2=`md5sum outSlow`
  if !(cmp -s "outWrong" "outSlow")
  then
      echo "Error found!"
      echo "Input:"
      cat stdinput
      echo "Wrong Output:"
      cat outWrong
      echo "Slow Output:"
      cat outSlow
      exit
  fi
done
echo Passed $4 tests
# ./stress.sh wrong correct gen times
```

7.9 vimrc [12 lines] - 1f47305c

```
"install xclip, vim-gtk3
set nocp ai bs=2 hls ic is lbr ls=2 mouse=a nu ru sc scs
    smd so=3 sw=2 ts=2 rnu
filetype plugin indent on
syn on
map gA m'ggVG"+y''
inoremap {<CR> {<CR>}<Esc>ko
nnoremap = mzgg=G`z
vnoremap <C-k> :s/^\s*\zs/\/\/ /<CR>:nohl<CR>
nnoremap <C-k> :s/^\s*\zs/\/\/ /<CR>:nohl<CR>
vnoremap <C-l> :s/^\s*\zs\/\/ //<CR>:nohl<CR>
nnoremap <C-l> :s/^\s*\zs\/\/ //<CR>:nohl<CR>
autocmd FileType cpp map <F9> :w<CR> :!clear; g++
    --std=c++17 % -DONPC -o %:r && ./%:r<CR>
```

8 Number Theory

8.1 10-ary to m-ary [15 lines] - 116e887b

```
// base conversion, 2 <= m <= 36, n >= 0
const string DIG =
    "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
string tenToM(ll n, int m) {
  if (n == 0) return "0";
  string res;
  while (n > 0) { res += DIG[n % m]; n /= m; }
  reverse(res.begin(), res.end());
  return res;
}
ll mToTen(const string& s, int m) {
  ll res = 0;
```

```
  for (char c : s) res = res * m + DIG.find(toupper(c));
  return res;
}
```

8.2 Basis Vector [90 lines] - e1218025

```
// T: int or ll, B: Max bits (31 for int, 62 for ll)
template<typename T = long long, int B = 62>
struct Basis {
  T basis[B];
  int sz, n; // sz: Num of independent vectors (rank)
  Basis() {
    memset(basis, 0, sizeof(basis));
    sz = 0; n = 0;
  }
  bool insert(T x) {
    n++;
    for (int i = B - 1; i >= 0; i--) {
      if (x >> i & 1) {
        if (!basis[i]) {
          basis[i] = x;
          sz++;
          return true;
        }
        x ^= basis[i];
      }
    }
    return false;
  }
  // Checks if any subset has XOR sum exactly x
  bool can(T x) {
    for (int i = B - 1; i >= 0; i--) {
      x = min(x, x ^ basis[i]);
    }
    return x == 0;
  }
  // Returns the maximum subset XOR sum
  T max_xor(T x = 0) {
    for (int i = B - 1; i >= 0; i--) {
      x = max(x, x ^ basis[i]);
      //x = min(x, x ^ basis[i]); //For min subset XOR
    }
    return x;
  }
  // k-th DISTINCT subset XOR sum in increasing order
  // (1-based, k=1 returns 0)
  T kth(unsigned long long k) {
    if (k < 1 || k > (1ULL << sz)) return -1;
    T x = 0;
    unsigned long long cnt = (1ULL << sz);
    for (int i = B - 1; i >= 0; i--) {
      if (basis[i]) {
        cnt >>= 1;
        if (k > cnt) {
          if (!(x >> i & 1)) x ^= basis[i]; // bit ON
          k -= cnt;
        } else {
          if (x >> i & 1) x ^= basis[i];    // bit OFF
        }
      }
    }
    return x;
  }
  // Count of DISTINCT subset XOR sums strictly < x
  unsigned long long count_lt(T x) {
    if (x < 0) return 0;
    unsigned long long ans = 0, cnt = (1ULL << sz);
    T mask = 0;
    for (int i = B - 1; i >= 0; i--) {
      if (basis[i]) {
        cnt >>= 1;
        if (x >> i & 1) {
          ans += cnt;
          if (!(mask >> i & 1)) mask ^= basis[i];
        } else {
          if (mask >> i & 1) mask ^= basis[i];
        }
      } else {
        if ((x >> i & 1) != (mask >> i & 1)) {
          return (x >> i & 1) ? ans + cnt : ans;
        }
      }
    }
    return ans;
  }
};
/* 1. SUBSET COUNTING MATH:
  - Valid combinations that XOR to 0: 2^(n - sz)
  - Valid non-empty combos that XOR to 0: 2^(n-sz) - 1
  - If can(X) is true, ways to form X: 2^(n - sz)
  2. DISTINCT VS TOTAL SUBSETS:
  - This struct works with DISTINCT XOR sums!
  - There are exactly (1ULL << sz) distinct XOR sums.
  - Each distinct XOR sum is formed by 2^(n-sz) subsets
  - If question asks for K-th out of ALL subsets:
    Pass: K = ceil(K / 2^(n - sz)) into kth() */
```

8.3 Binary Exponentiation [12 lines] - 7916ccff

```
// b^e % mod. mod is REQUIRED (no default = no wrong-
// mod WA)
// __int128 makes it safe for mod up to ~1e18
ll power(ll b, ll e, ll mod) {
  ll r = 1 % mod; b %= mod; if (b < 0) b += mod;
  while (e > 0) {
    if (e & 1) r = (__int128)r * b % mod;
    b = (__int128)b * b % mod;
    e >>= 1;
  }
  return r;
}
```

8.4 CRT [20 lines] - f680196e

```
// x = r1 (mod m1), x = r2 (mod m2); needs 0 <= r < m
// returns smallest x >= 0 (mod lcm), -1 if no solution
// uses exgcd (Extended GCD). lcm must fit in ll
ll crt(ll r1, ll m1, ll r2, ll m2) {
  if (m1 < m2) swap(r1, r2), swap(m1, m2);
  ll p, q, g = exgcd(m1, m2, p, q);
  if ((r2 - r1) % g != 0) return -1;  // no solution
  __int128 t = (__int128)((r2 - r1) % m2) * p % m2;
  ll x = (ll)(t * m1 / g) + r1;
  return x < 0 ? x + m1 / g * m2 : x;
}
ll crt(vector<ll>& r, vector<ll>& m) {
  ll x = r[0], M = m[0];
  for (int i = 1; i < (int)r.size(); ++i) {
    x = crt(x, M, r[i], m[i]);
    if (x == -1) return -1;
    M = M / __gcd(M, m[i]) * m[i];
  }
  return x;
}
```

8.5 Count 1s 0 to n [9 lines] - 9292bf04

```
int cntOnes(int n) {
  int cnt = 0;
  for (int i = 1; i <= n; i <<= 1) {
    int x = (n + 1) / (i << 1);
    cnt += x * i;
    if ((n + 1) % i && n & i) cnt += (n + 1) % i;
  }
  return cnt;
}
```

8.6 Divisors multiple info [26 lines] - a222a3d7

```
// Factorize n with `primes` (Sieve, up to sqrt n):
// min/max prime, #distinct primes, #primes with power,
// NOD (number of divisors), SOD (sum of divisors)
void divisorsInfo(ll n, vector<int>& primes) {
  ll nod = 1, sod = 1, mnP = LLONG_MAX, mxP = 0;
  int cntP = 0, totalP = 0;
  for (ll p : primes) {
    if (p * p > n) break;
    if (n % p) continue;
    mnP = min(mnP, p); mxP = max(mxP, p);
    int k = 0; ll s = 1, pw = 1;
    while (n % p == 0) {
      n /= p; k++; pw *= p; s += pw;
    }
    cntP++; totalP += k;
    nod *= (k + 1);
    sod *= s;
  }
  if (n > 1) { // remaining prime factor
    cntP++, totalP++;
    nod *= 2; sod *= (1 + n);
    mnP = min(mnP, n); mxP = max(mxP, n);
  }
  cout << mnP << " " << mxP << " " << cntP << " "
       << totalP << " " << nod << " " << sod << "\n";
}
```

8.7 Extended GCD [12 lines] - 294ded67

```
// returns g = gcd(a, b), finds x, y with a*x + b*y = g
ll exgcd(ll a, ll b, ll &x, ll &y) {
  if (b == 0) {
    x = 1; y = 0;
    return a;
  }
  ll g = exgcd(b, a % b, y, x);
  y -= a / b * x;
  return g;
}
// all solutions: x + k*(b/g), y - k*(a/g)
// a*x + b*y = c solvable iff c % g == 0 (scale by c/g)
```

8.8 GaussElimination [40 lines] - 3f353a27

```
template<typename ld>
int gauss(vector<vector<ld>>& a, vector<ld>& ans) {
  const ld EPS = 1e-9;
  int n = a.size();///number of equations
  int m = a[0].size() - 1;///number of variables
  // where[col]: which row contains the solution
  vector<int>where(m, -1);
  int row, col;
  for (col = 0, row = 0;col < m && row < n;++col) {
    int sel = row; // row containing the maximum value
    for (int i = row + 1;i < n;i++)
      if (abs(a[i][col]) > abs(a[sel][col]))
        sel = i;
```

```
    if (abs(a[sel][col]) < EPS) continue; // ~0
    a[sel].swap(a[row]);///taking the max row up
    where[col] = row;
    ld t = a[row][col];
    for (int i = col;i <= m;i++) a[row][i] /= t;
    for (int i = 0;i < n;i++) {
      if (i != row) {
        ld c = a[i][col];
        for (int j = col;j <= m;j++)
          a[i][j] -= a[row][j] * c;
      }
    }
    row++;
  }
  ans.assign(m, 0);
  for (int i = 0;i < m;i++)
    if (where[i] != -1)
      ans[i] = a[where[i]][m] / a[where[i]][i];
  for (int i = 0;i < n;i++) {
    ld sum = 0;
    for (int j = 0;j < m;j++)
      sum += ans[j] * a[i][j];
    if (abs(sum - a[i][m]) > EPS) ///L.H.S!=R.H.S
      ans.clear();//No solution
  }
  return row;
}
```

8.9 Lagrange Interpolation [21 lines] - b65ceb53

```
// P has degree <= n, known y[i] = P(i) for i = 0..n.
// Returns P(x) mod p in O(n) (needs fact / ifact from
// nCr). e.g. sum_{i=1..x} i^k is degree k+1 in x
ll lagrange(vector<ll>& y, ll x) {
  int n = y.size() - 1;
  x %= mod; if (x < 0) x += mod;
  if (x <= n) return y[x];
  vector<ll> pre(n + 2, 1), suf(n + 2, 1);
  for (int i = 0; i <= n; i++)
    pre[i + 1] = pre[i] * ((x - i) % mod) % mod;
  for (int i = n; i >= 0; i--)
    suf[i] = suf[i + 1] * ((x - i) % mod) % mod;
  ll res = 0;
  for (int i = 0; i <= n; i++) {
    ll term = y[i] * pre[i] % mod * suf[i + 1] % mod
              * ifact[i] % mod * ifact[n - i] % mod;
    if ((n - i) & 1) term = (mod - term) % mod;
    res = (res + term) % mod;
  }
  return res;
}
```

8.10 Largest Power of n! [10 lines] - 003e5b0e

```
// Returns largest power of p that divides n!
int largestPower(int n, int p)  {
  int res = 0;
  // Calculate res = n/p + n/(p^2) + n/(p^3) + ....
  while (n > 0) {
    n /= p;
    res += n;
  }
  return res;
}
```

8.11 Leap Year [15 lines] - 4af3cf4a

```
bool isLeap(int n){
  if (n%100==0)
    if (n%400==0) return true;
    else return false;
  if (n%4==0) return true;
  else return false;
}
// Num of Leap year in between
int calNum(int year) {
  return (year / 4) - (year / 100) +
  (year / 400);
}
int leapNum(int l, int r) {
  return calNum(r) - calNum(--l);
}
```

8.12 Linear Sieve (SPF, Phi, Mobius) [35 lines] - edb0484a

```
// O(N): primes, smallest prime factor, phi, Mobius
// factorize x < N in O(log x) by repeatedly x /=
    spf[x]
const int N = 1e6 + 5;
int spf[N], phi[N], mu[N];
vector<int> primes;
void linearSieve() {
  phi[1] = mu[1] = 1;
  for (int i = 2; i < N; i++) {
    if (!spf[i]) {
      spf[i] = i; phi[i] = i - 1; mu[i] = -1;
      primes.push_back(i);
    }
    for (int p : primes) {
      if (p > spf[i] || 1LL * i * p >= N) break;
      spf[i * p] = p;
      if (p == spf[i]) { // p^2 divides i * p
        phi[i * p] = phi[i] * p; mu[i * p] = 0;
      } else {
        phi[i * p] = phi[i] * (p - 1);
        mu[i * p] = -mu[i];
      }
    }
  }
}
// phi of one number, O(sqrt n)
ll phiOf(ll n) {
  ll r = n;
  for (ll p = 2; p * p <= n; p++)
    if (n % p == 0) {
      while (n % p == 0) n /= p;
      r -= r / p;
    }
  if (n > 1) r -= r / n;
  return r;
}
```

8.13 Matrix [46 lines] - f0928c7e

```
// Matrix mod MOD (real matrix / inverse: use Gauss)
const ll MOD = 1e9 + 7; // change if necessary
struct Matrix {
  int R, C;
  vector<vector<ll>> mat;
  Matrix(int R, int C, bool identity = false)
      : R(R), C(C), mat(R, vector<ll>(C, 0)) {
    if (identity)
      for (int i = 0; i < R; i++) mat[i][i] = 1;
  }
  Matrix operator+(const Matrix& o) const {
    Matrix res(R, C);
    for (int i = 0; i < R; i++)
      for (int j = 0; j < C; j++)
        res.mat[i][j] = (mat[i][j] + o.mat[i][j]) % MOD;
    return res;
  }
  Matrix operator-(const Matrix& o) const {
    Matrix res(R, C);
    for (int i = 0; i < R; i++)
      for (int j = 0; j < C; j++)
        res.mat[i][j] =
            (mat[i][j] - o.mat[i][j] + MOD) % MOD;
    return res;
  }
  Matrix operator*(const Matrix& o) const {
    Matrix res(R, o.C);
    for (int i = 0; i < R; i++)
      for (int k = 0; k < C; k++) {
        if (mat[i][k] == 0) continue;
        for (int j = 0; j < o.C; j++)
          res.mat[i][j] = (res.mat[i][j] +
              mat[i][k] * o.mat[k][j]) % MOD;
      }
    return res;
  }
  Matrix pow(ll e) const { // square matrix only
    Matrix res(R, R, true), b = *this;
    for (; e > 0; e >>= 1, b = b * b)
      if (e & 1) res = res * b;
    return res;
  }
};
// Fibonacci: Matrix base(2, 2);
// base.mat = {{1, 1}, {1, 0}};
// F(n) = base.pow(n).mat[0][1]    (F(0) = 0, F(1) = 1)
```

8.14 Miller-Rabin-Pollard-Rho [55 lines] - 212eb3a0

```
// isPrime: deterministic for all n < 2^64
// factor(n): prime factors (unsorted, with repeats)
using u64 = uint64_t; using u128 = __uint128_t;
u64 mulmod(u64 a, u64 b, u64 m) {
  return (u128)a * b % m;
}
u64 powmod(u64 b, u64 e, u64 m) {
  u64 r = 1; b %= m;
  for (; e; e >>= 1, b = mulmod(b, b, m))
    if (e & 1) r = mulmod(r, b, m);
  return r;
}
bool isPrime(u64 n) {
  if (n < 2) return false;
  for (u64 p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
                37})
    if (n % p == 0) return n == p;
  u64 d = n - 1; int s = __builtin_ctzll(d); d >>= s;
  for (u64 a : {2, 325, 9375, 28178, 450775, 9780504,
                1795265022}) {
    u64 x = powmod(a, d, n);
    if (x == 0 || x == 1 || x == n - 1) continue;
    bool composite = true;
    for (int r = 1; r < s && composite; r++) {
      x = mulmod(x, x, n);
      if (x == n - 1) composite = false;
    }
    if (composite) return false;
  }
  return true;
}
mt19937_64 rng64(chrono::steady_clock::now()
```

```
                    .time_since_epoch().count());
// a non-trivial factor of composite n
u64 pollard(u64 n) {
  if (n % 2 == 0) return 2;
  while (true) {
    u64 x = rng64() % (n - 2) + 2, y = x;
    u64 c = rng64() % (n - 1) + 1, d = 1;
    auto f = [&](u64 v) {
      return (mulmod(v, v, n) + c) % n;
    };
    while (d == 1) {
      x = f(x); y = f(f(y));
      d = __gcd(x > y ? x - y : y - x, n);
    }
    if (d != n) return d;
  }
}
void factor(u64 n, vector<u64>& f) {
  if (n == 1) return;
  if (isPrime(n)) { f.push_back(n); return; }
  u64 d = pollard(n);
  factor(d, f); factor(n / d, f);
}
```

8.15 Mod Inverse [11 lines] - fb916c29

```
// a^-1 mod m for ANY m with gcd(a, m) = 1 (uses exgcd)
// returns -1 if inverse doesn't exist
// m prime: power(a, m - 2, m) also works
ll modInv(ll a, ll m) {
  ll x, y;
  ll g = exgcd(((a % m) + m) % m, m, x, y);
  if (g != 1) return -1;
  return (x % m + m) % m;
}
// all inverses 1..n mod prime p in O(n):
// inv[1] = 1; inv[i] = (p - p / i) * inv[p % i] % p
```

8.16 Modint [34 lines] - b857de88

```
// Auto-mod arithmetic. MOD must be PRIME for division
template <int MOD> struct Mint {
  int v;
  Mint(ll x = 0) { v = x % MOD; if (v < 0) v += MOD; }
  Mint& operator+=(Mint o) {
    if ((v += o.v) >= MOD) v -= MOD;
    return *this;
  }
  Mint& operator-=(Mint o) {
    if ((v -= o.v) < 0) v += MOD;
    return *this;
  }
  Mint& operator*=(Mint o) {
    v = 1LL * v * o.v % MOD;
    return *this;
  }
  Mint pow(ll e) const {
    Mint r = 1, b = *this;
    for (; e > 0; e >>= 1, b *= b) if (e & 1) r *= b;
    return r;
  }
  Mint inv() const { return pow(MOD - 2); }
  Mint& operator/=(Mint o) { return *this *= o.inv(); }
  friend Mint operator+(Mint a, Mint b) {return a += b;}
  friend Mint operator-(Mint a, Mint b) {return a -= b;}
  friend Mint operator*(Mint a, Mint b) {return a *= b;}
  friend Mint operator/(Mint a, Mint b) {return a /= b;}
  bool operator==(Mint o) const { return v == o.v; }
  friend ostream& operator<<(ostream& os, Mint a) {
    return os << a.v;
  }
};
using mint = Mint<1000000007>; // or 998244353
// mint a = 5, b = 3; cout << a / b << a.pow(10);
```

8.17 NOD of n! [13 lines] - 60fcf48f

```
ull factorialDivisors(ull n) {
  ull result = 1;
  for (int i = 0; i < allPrimes.size(); i++) {
    ull p = allPrimes[i];
    ull exp = 0;
    while (p <= n) {
      exp = exp + (n / p);
      p = p * allPrimes[i];
    }
    result = result * (exp + 1);
  }
  return result;
}
```

8.18 NTT and FFT [85 lines] - 83a7fe8f

```
// Polynomial multiplication O(n log n)
// NTT: exact mod 998244353 (root 3). Any mod: FFT with
// 15-bit split, or CRT of 3 NTT primes.
const int MODN = 998244353, ROOT = 3;
ll pw(ll b, ll e) {
  ll r = 1; b %= MODN;
  for (; e; e >>= 1, b = b * b % MODN)
    if (e & 1) r = r * b % MODN;
  return r;
}
void ntt(vector<ll>& a, bool inv) {
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    ll w = pw(ROOT, (MODN - 1) / len);
    if (inv) w = pw(w, MODN - 2);
    for (int i = 0; i < n; i += len) {
      ll wn = 1;
      for (int j = 0; j < len / 2; j++) {
        ll u = a[i + j];
        ll v = a[i + j + len / 2] * wn % MODN;
        a[i + j] = (u + v) % MODN;
        a[i + j + len / 2] = (u - v + MODN) % MODN;
        wn = wn * w % MODN;
      }
    }
  }
  if (inv) {
    ll ni = pw(n, MODN - 2);
    for (ll& x : a) x = x * ni % MODN;
  }
}
vector<ll> multiply(vector<ll> a, vector<ll> b) {
  if (a.empty() || b.empty()) return {};
  int need = a.size() + b.size() - 1, n = 1;
  while (n < need) n <<= 1;
  a.resize(n); b.resize(n);
  ntt(a, false); ntt(b, false);
  for (int i = 0; i < n; i++) a[i] = a[i] * b[i] % MODN;
  ntt(a, true); a.resize(need);
  return a;
}
// FFT (doubles): exact if result coefficients < ~1e15
typedef complex<double> cd;
void fft(vector<cd>& a, bool inv) {
  int n = a.size();
  for (int i = 1, j = 0; i < n; i++) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    double ang = 2 * acos(-1) / len * (inv ? -1 : 1);
    cd w(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
      cd wn(1);
      for (int j = 0; j < len / 2; j++, wn *= w) {
        cd u = a[i + j], v = a[i + j + len / 2] * wn;
        a[i + j] = u + v; a[i + j + len / 2] = u - v;
      }
    }
  }
  if (inv) for (cd& x : a) x /= n;
}
vector<ll> multiplyFFT(vector<ll>& a, vector<ll>& b) {
  if (a.empty() || b.empty()) return {};
  int need = a.size() + b.size() - 1, n = 1;
  while (n < need) n <<= 1;
  vector<cd> fa(a.begin(), a.end());
  vector<cd> fb(b.begin(), b.end());
  fa.resize(n); fb.resize(n);
  fft(fa, false); fft(fb, false);
  for (int i = 0; i < n; i++) fa[i] *= fb[i];
  fft(fa, true);
  vector<ll> res(need);
  for (int i = 0; i < need; i++)
    res[i] = llround(fa[i].real());
  return res;
}
```

8.19 No of Digits in n! in base B [7 lines] - 9ab8b6e6

```
// digits of n! in base B = floor(log_B(n!)) + 1, O(1)
// lgamma(n + 1) = ln(n!). eps guards exact powers
ll digitsOfFact(ll n, ll B) {
  if (n <= 1) return 1;
  long double lg = lgamma(n + 1.0L) / logl(B);
  return (ll)floorl(lg + 1e-12L) + 1;
}
```

8.20 Principle of Inclusion Exclusion [19 lines] - bd41e547

```
// count numbers in [1, num] NOT divisible by any of
// div[0..n-1] = num - (|A| - |A&B| + |A&B&C| - ...)
ll PIE(vector<ll>& div, ll num) {
  int n = div.size();
  ll sum = 0;
  for (int msk = 1; msk < (1 << n); ++msk) {
    ll lcm = 1;
    for (int i = 0; i < n && lcm <= num; ++i)
      if (msk >> i & 1) {
        ll g = __gcd(lcm, div[i]);
        if (lcm / g > num / div[i]) lcm = num + 1;
        else lcm = lcm / g * div[i];
      }
    ll cur = num / lcm;
```

```
    if (__builtin_popcount(msk) & 1) sum += cur;
    else sum -= cur;
  }
  return num - sum;
}
```

8.21 Print Calander of Any Year [38 lines] - 0fdba063

```
// 0 = Sunday, ..., 6 = Saturday. month is 1-based here
int dayNumber(int day, int month, int year) {
  static int t[] = {0, 3, 2, 5, 0, 3, 5, 1, 4, 6, 2, 4};
  year -= month < 3;
  return (year + year / 4 - year / 100 + year / 400
          + t[month - 1] + day) % 7;
}
string getMonthName(int month) { // month is 0-based
  string months[] = {"January", "February", "March",
    "April", "May", "June", "July", "August",
    "September", "October", "November", "December"};
  return months[month];
}
// isLeap() from Leap Year
int numberOfDays(int month, int year) {
  if (month == 1 && isLeap(year)) return 29;
  int days[] = {31, 28, 31, 30, 31, 30,
                31, 31, 30, 31, 30, 31};
  return days[month];
}
void printCalendar(int year) {
  printf("       Calendar - %d\n\n", year);
  int current = dayNumber(1, 1, year);
  for (int i = 0; i < 12; i++) { // i -> months
    int days = numberOfDays(i, year);
    cout << "         |" << getMonthName(i) << "|\n";
    printf(" Sun Mon Tue Wed Thu Fri Sat\n");
    int k;
    for (k = 0; k < current; k++) printf("    ");
    for (int j = 1; j <= days; j++) { // j -> days
      printf("%4d", j);
      if (++k > 6) { k = 0; cout << endl; }
    }
    if (k) cout << endl;
    cout << "---------------------------\n";
    current = k;
  }
}
```

8.22 SOD, NOD Upto N [25 lines] - d2b85209

```
// sum_{i=1..N} SOD(i) = sum_i i * (N / i). O(sqrt N)
// result ~ 0.82 * N^2 -> fits ll for N <= ~3e9
ll SOD_upto_N(ll N) {
  ll res = 0;
  for (ll i = 1, j; i <= N; i = j + 1) {
    j = N / (N / i); // last i with the same quotient
    res += (N / i) * ((i + j) * (j - i + 1) / 2);
  }
  return res;
}
// sum_{i=1..N} NOD(i) = sum_i (N / i). O(sqrt N)
ll NOD_upto_N(ll N) {
  ll res = 0, u = sqrtl(N);
  while (u * u > N) u--;
  while ((u + 1) * (u + 1) <= N) u++;
  for (ll i = 1; i <= u; i++) res += N / i - i;
  return res * 2 + u;
}
// NOD / SOD of every number 1..n. O(n log n)
vector<ll> divisorSieve(int n, bool sod = false) {
  vector<ll> d(n + 1, 0);
  for (int i = 1; i <= n; i++)
    for (int j = i; j <= n; j += i) d[j] += sod ? i : 1;
  return d;
}
```

8.23 Segmented Sieve [16 lines] - 88be0e97

```
// Primes in [L, R], R up to ~1e12, R - L up to ~1e7
// `primes` must contain all primes <= sqrt(R) (Sieve)
vector<ll> segmentedSieve(ll L, ll R,
                          vector<int>& primes) {
  vector<bool> isPrime(R - L + 1, true);
  for (ll p : primes) {
    if (p * p > R) break;
    ll start = max(p * p, (L + p - 1) / p * p);
    for (ll j = start; j <= R; j += p)
      isPrime[j - L] = false;
  }
  vector<ll> res;
  for (ll i = max(L, 2LL); i <= R; i++)
    if (isPrime[i - L]) res.push_back(i);
  return res;
}
```

8.24 Sieve upto 1e9 [75 lines] - dc302c09

```
// returns all primes <= N (N up to 1e9) in ~1s
vector<int> sieve(const int N, const int Q = 17,
                  const int L = 1 << 15) {
  static const int rs[] =
      {1, 7, 11, 13, 17, 19, 23, 29};
  struct P {
    P(int p) : p(p) {}
    int p; int pos[8];
  };
  auto approx_cnt = [] (const int N) -> int {
    return N > 60184 ? N / (log(N) - 1.1)
                     : max(1., N / (log(N) - 1.11)) + 1;
  };
  const int v = sqrt(N), vv = sqrt(v);
  vector<bool> isp(v + 1, true);
  for (int i = 2; i <= vv; ++i) if (isp[i]) {
    for (int j = i * i; j <= v; j += i) isp[j] = false;
  }
  const int rsize = approx_cnt(N + 30);
  vector<int> primes = {2, 3, 5}; int psize = 3;
  primes.resize(rsize);
  vector<P> sprimes; size_t pbeg = 0;
  int prod = 1;
  for (int p = 7; p <= v; ++p) {
    if (!isp[p]) continue;
    if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
    auto pp = P(p);
    for (int t = 0; t < 8; ++t) {
      int j = (p <= Q) ? p : p * p;
      while (j % 30 != rs[t]) j += p << 1;
      pp.pos[t] = j / 30;
    }
    sprimes.push_back(pp);
  }
  vector<unsigned char> pre(prod, 0xFF);
  for (size_t pi = 0; pi < pbeg; ++pi) {
    auto pp = sprimes[pi]; const int p = pp.p;
    for (int t = 0; t < 8; ++t) {
      const unsigned char m = ~(1 << t);
      for (int i = pp.pos[t]; i < prod; i += p)
        pre[i] &= m;
    }
  }
  const int bsz = (L + prod - 1) / prod * prod;
  vector<unsigned char> block(bsz);
  unsigned char* pblock = block.data();
  const int M = (N + 29) / 30;
  for (int beg = 0; beg < M;
       beg += bsz, pblock -= bsz) {
    int end = min(M, beg + bsz);
    for (int i = beg; i < end; i += prod) {
      copy(pre.begin(), pre.end(), pblock + i);
    }
    if (beg == 0) pblock[0] &= 0xFE;
    for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
      auto& pp = sprimes[pi];
      const int p = pp.p;
      for (int t = 0; t < 8; ++t) {
        int i = pp.pos[t];
        const unsigned char m = ~(1 << t);
        for (; i < end; i += p) pblock[i] &= m;
        pp.pos[t] = i;
      }
    }
    for (int i = beg; i < end; ++i) {
      for (int m = pblock[i]; m > 0; m &= m - 1) {
        primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
      }
    }
  }
  assert(psize <= rsize);
  while (psize > 0 && primes[psize - 1] > N) --psize;
  primes.resize(psize);
  return primes;
}
```

8.25 Sieve [12 lines] - 66842211

```
// primes < N. bitset: 1e7 -> 1.2 MB (int array: 40 MB)
const int N = 1e7 + 3;
vector<int> primes;
bitset<N> notprime;
void sieve() {
  notprime[0] = notprime[1] = 1;
  for (ll i = 2; i < N; i++) {
    if (notprime[i]) continue;
    primes.push_back(i);
    for (ll j = i * i; j < N; j += i) notprime[j] = 1;
  }
}
```

8.26 String Mod [6 lines] - 2b0c0c9d

```
// (big number given as string) % a
ll strMod(const string& num, ll a) {
  ll res = 0;
  for (char c : num) res = (res * 10 + (c - '0')) % a;
  return res;
}
```

8.27 nCr [23 lines] - 8294a19d

```
// nCr mod prime in O(1) after O(N) precompute
const int N = 1e6 + 5, mod = 1e9 + 7;
ll fact[N], ifact[N];
void preFact() {
  fact[0] = 1;
  for (int i = 1; i < N; i++)
    fact[i] = fact[i - 1] * i % mod;
  ifact[N - 1] = power(fact[N - 1], mod - 2, mod);
```

```
  for (int i = N - 2; i >= 0; i--)
    ifact[i] = ifact[i + 1] * (i + 1) % mod;
}
ll nCr(int n, int r) {
  if (r < 0 || n < r) return 0;
  return fact[n] * ifact[r] % mod * ifact[n - r] % mod;
}
ll nPr(int n, int r) {
  if (r < 0 || n < r) return 0;
  return fact[n] * ifact[n - r] % mod;
}
// Catalan(n) = nCr(2n, n) * inv(n + 1)
//         = (nCr(2n, n) - nCr(2n, n + 1) + mod) % mod
// Stars & bars: x1+..+xk = n, xi >= 0
//               -> nCr(n + k - 1, k - 1)
```

9 Some Algorithms

9.1 2D Prefix Sum [4 lines] - 3c8700da

```
pref[i][j] = a[i][j] + pref[i - 1][j] + pref[i][j - 1]
           - pref[i - 1][j - 1];
Sum of region = pref[row2][col2] - pref[row2][col1 - 1]
    - pref[row1 - 1][col2] + pref[row1 - 1][col1 - 1];
```

9.2 Big Integer [100 lines] - 72f848e2

```
// Non-negative big integers as decimal strings
struct BigInteger {
  string str;
  // Constructor to initialize
  // BigInteger with a string
  BigInteger(string s) { str = s; }
  // Overload + operator to add
  // two BigInteger objects
  BigInteger operator+(const BigInteger& b) {
    string a = str, c = b.str;
    int alen = a.length(), clen = c.length();
    int n = max(alen, clen);
    if (alen > clen)
      c.insert(0, alen - clen, '0');
    else if (alen < clen)
      a.insert(0, clen - alen, '0');
    string res(n + 1, '0');
    int carry = 0;
    for (int i = n - 1; i >= 0; i--) {
      int digit = (a[i] - '0') + (c[i] - '0') + carry;
      carry = digit / 10;
      res[i + 1] = digit % 10 + '0';
    }
    if (carry == 1) {
      res[0] = '1';
      return BigInteger(res);
    } else
      return BigInteger(res.substr(1));
  }
  // returns |a - b|
  BigInteger operator-(const BigInteger& b) {
    string a = str;
    string c = b.str;
    int alen = a.length(), clen = c.length();
    int n = max(alen, clen);
    if (alen > clen)
      c.insert(0, alen - clen, '0');
    else if (alen < clen)
      a.insert(0, clen - alen, '0');
    if (a < c) {
      swap(a, c);
      swap(alen, clen);
    }
    string res(n, '0');
    int carry = 0;
    for (int i = n - 1; i >= 0; i--) {
      int digit = (a[i] - '0') - (c[i] - '0') - carry;
      if (digit < 0)
        digit += 10, carry = 1;
      else
        carry = 0;
      res[i] = digit + '0';
    }
    // remove leading zeros
    int i = 0;
    while (i < n && res[i] == '0') i++;
    if (i == n) return BigInteger("0");
    return BigInteger(res.substr(i));
  }
  // Overload * operator to multiply
  // two BigInteger objects
  BigInteger operator*(const BigInteger& b) {
    string a = str, c = b.str;
    int alen = a.length(), clen = c.length();
    int n = alen + clen;
    string res(n, '0');
    for (int i = alen - 1; i >= 0; i--) {
      int carry = 0;
      for (int j = clen - 1; j >= 0; j--) {
        int digit = (a[i] - '0') * (c[j] - '0')
                    + (res[i + j + 1] - '0') + carry;
        carry = digit / 10;
        res[i + j + 1] = digit % 10 + '0';
      }
      res[i] += carry;
    }
    int i = 0;
    while (i < n && res[i] == '0') i++;
    if (i == n) return BigInteger("0");
    return BigInteger(res.substr(i));
  }
  // divide by small d > 0, r = remainder
  BigInteger divSmall(ll d, ll& r) const {
    string q; r = 0;
    for (char ch : str) {
      r = r * 10 + (ch - '0');
      q += char('0' + r / d); r %= d;
    }
    size_t k = q.find_first_not_of('0');
    return BigInteger(k == string::npos ? "0"
                                        : q.substr(k));
  }
  // Overload << operator to output
  // BigInteger object
  friend ostream& operator<<(ostream& out,
                             const BigInteger& b) {
    out << b.str;
    return out;
  }
};
```

9.3 Expression Parsing [90 lines] - e4cc53dd

```
bool delim(char c) { return c == ' '; }
bool is_op(char c) {
  return c == '+' || c == '-' || c == '*' || c == '/';
}
bool is_unary(char c) { return c == '+' || c == '-'; }
int priority(char op) {
  if (op < 0)  // unary operator
    return 3;
  if (op == '+' || op == '-') return 1;
  if (op == '*' || op == '/') return 2;
  return -1;
}
void process_op(stack<int>& st, char op) {
  if (op < 0) {
    int l = st.top();
    st.pop();
    switch (-op) {
      case '+':
        st.push(l);
        break;
      case '-':
        st.push(-l);
        break;
    }
  } else {
    int r = st.top();
    st.pop();
    int l = st.top();
    st.pop();
    switch (op) {
      case '+':
        st.push(l + r);
        break;
      case '-':
        st.push(l - r);
        break;
      case '*':
        st.push(l * r);
        break;
      case '/':
        st.push(l / r);
        break;
    }
  }
}
int evaluate(string& s) {
  stack<int> st;
  stack<char> op;
  bool may_be_unary = true;
  for (int i = 0; i < (int)s.size(); i++) {
    if (delim(s[i])) continue;
    if (s[i] == '(') {
      op.push('(');
      may_be_unary = true;
    } else if (s[i] == ')') {
      while (op.top() != '(') {
        process_op(st, op.top());
        op.pop();
      }
      op.pop();
      may_be_unary = false;
    } else if (is_op(s[i])) {
      char cur_op = s[i];
      if (may_be_unary && is_unary(cur_op))
        cur_op = -cur_op;
      while (!op.empty() &&
        ((cur_op >= 0 &&
          priority(op.top()) >= priority(cur_op)) ||
         (cur_op < 0 &&
          priority(op.top()) > priority(cur_op)))) {
        process_op(st, op.top());
```

```
      op.pop();
    }
    op.push(cur_op);
    may_be_unary = true;
  } else {
    int number = 0;
    while (i < (int)s.size() && isalnum(s[i]))
      number = number * 10 + s[i++] - '0';
    --i;
    st.push(number);
    may_be_unary = false;
  }
}
while (!op.empty()) {
  process_op(st, op.top());
  op.pop();
}
return st.top();
}
```

9.4 Kadanes [9 lines] - 8b61706c

```
// return maximum subarray sum.
int kadanes(int arr[], int n) {
  int maxsum = arr[0], curr_sum = arr[0];
  for (int i = 1; i < n; i++) {
    curr_sum = max(arr[i], curr_sum + arr[i]);
    maxsum = max(maxsum, curr_sum);
  }
  return maxsum;
}
```

9.5 Monotonic Stack (next greater) [8 lines] - c63e64cb

```
for (int i = n - 1; i >= 0; i--) {
  while (!stk.empty() && v[i] >= v[stk.top()])
    stk.pop();
  ind[i] = stk.empty() ? -1 : stk.top();
  stk.push(i);
}
// 3 1 5 4 10
// 2 2 4 4 -1
```

9.6 Ternary Search [10 lines] - c9abc776

```
double ternary_search(double l, double r) {
  double eps = 1e-9;  // error limit
  while (r - l > eps) {
    double m1 = l + (r - l) / 3, m2 = r - (r - l) / 3;
    double f1 = f(m1), f2 = f(m2);  // f at m1, m2
    if (f1 < f2) l = m1;
    else r = m2;
  }
  return f(l);  // return the maximum of f(x) in [l, r]
}
```

10 String

10.1 AhoCorasick [56 lines] - ce3475dc

```
struct AhoCorasick {
  struct Node {
    int nxt[26]{};
    int fail = 0, term = 0, cnt = 0, id = -1;
  };
  vector<Node> t;
  AhoCorasick() : t(1) {} // Root is at index 0
  void insert(const string& s, int idx) {
    int u = 0;
    for (char c : s) {
      int v = c - 'a';
      if (!t[u].nxt[v]) {
        t[u].nxt[v] = t.size();
        t.emplace_back();
      }
      u = t[u].nxt[v];
    }
    t[u].cnt++;
    t[u].id = idx;
  }
  void build() {
    queue<int> q;
    for (int i = 0; i < 26; i++) {
      if (t[0].nxt[i]) q.push(t[0].nxt[i]);
    }
    while (!q.empty()) {
      int u = q.front(); q.pop();
      t[u].cnt += t[t[u].fail].cnt;
      t[u].term = (t[t[u].fail].id != -1)
                  ? t[u].fail : t[t[u].fail].term;
      for (int i = 0; i < 26; i++) {
        if (t[u].nxt[i]) {
          t[t[u].nxt[i]].fail = t[t[u].fail].nxt[i];
          q.push(t[u].nxt[i]);
        } else {
          t[u].nxt[i] = t[t[u].fail].nxt[i];
        }
      }
    }
  }
  long long matches(const string& txt) {
    long long matches = 0;
    int u = 0;
    for(int i = 0;i<txt.size();i++){
      u = t[u].nxt[txt[i] - 'a'];
      matches += t[u].cnt;
      int cur = u;
      if(t[cur].id==-1)cur = t[cur].term;
      while(cur>0){
        //exact match
        cur = t[cur].term;
      }
    }
    return matches;
  }
};
```

10.2 Hashing [51 lines] - 05c75f15

```
const int N = 2e6 + 9;
const int M1 = 1e9 + 7, M2 = 1e9 + 9;
const int B1 = 137, B2 = 277;
pair<int, int> pw[N];
void init_hash() {
  pw[0] = {1, 1};
  for (int i = 1; i < N; i++) {
    pw[i].first = 1LL * pw[i - 1].first * B1 % M1;
    pw[i].second = 1LL * pw[i - 1].second * B2 % M2;
  }
}
struct Hash {
  string s;
  vector<pair<int, int>> hs;
  Hash(const string& _s) : s(_s) {
    hs.assign(s.size() + 1, {0, 0});
    for (int i = 0; i < s.size(); i++) {
      hs[i + 1].first =
          (1LL * hs[i].first * B1 + s[i]) % M1;
      hs[i + 1].second =
          (1LL * hs[i].second * B2 + s[i]) % M2;
    }
  }
  pair<int, int> get(int l, int r) { // hash of s[l..r]
    int h1 = (hs[r + 1].first - 1LL * hs[l].first
              * pw[r - l + 1].first % M1 + M1) % M1;
    int h2 = (hs[r + 1].second - 1LL * hs[l].second
              * pw[r - l + 1].second % M2 + M2) % M2;
    return {h1, h2};
  }
  int lcp(int i1, int j1, int i2, int j2) {
    int l = 1, r = min(j1 - i1 + 1, j2 - i2 + 1);
    int ans = 0;
    while (l <= r) {
      int mid = (l + r) / 2;
      if (get(i1, i1 + mid - 1) ==
          get(i2, i2 + mid - 1))
        ans = mid, l = mid + 1;
      else r = mid - 1;
    }
    return ans;
  }
  int compare(int i1, int j1, int i2, int j2) {
    int l = lcp(i1, j1, i2, j2);
    int len1 = j1 - i1 + 1, len2 = j2 - i2 + 1;
    if (l == len1 && l == len2) return 0;
    if (l == len1) return -1;
    if (l == len2) return 1;
    return s[i1 + l] < s[i2 + l] ? -1 : 1;
  }
};
```

10.3 KMP [29 lines] - 6273b948

```
struct KMP {
  string s; // must declare s before n
  int n; vector<int> pi;
  KMP(const string& _s) : s(_s), n(s.size()), pi(n, 0) {
    for (int i = 1, j = 0; i < n; i++) {
      while (j > 0 && s[i] != s[j]) j = pi[j - 1];
      if (s[i] == s[j]) j++;
      pi[i] = j;
    }
  }
  // automaton: nxt[state][char] -> next state
  vector<vector<int>> build_nxt(int A = 26,
                                char base = 'a') {
    vector<vector<int>> nxt(n + 1, vector<int>(A, 0));
    for (int i = 0; i <= n; i++)
      for (int c = 0; c < A; c++)
        if (i < n && s[i] - base == c)
          nxt[i][c] = i + 1;
        else if (i > 0) nxt[i][c] = nxt[pi[i - 1]][c];
    return nxt;
  }
};
// MATCH: KMP k(pat + "#" + txt); int m = pat.size();
// for(int i = m+1; i < k.n; i++) if(k.pi[i] == m)
//   { /* Match starts at txt[i - 2*m] */ }
// STR DP: auto nxt = KMP(pat).build_nxt();
// states: 0 to m. (st == m means fully matched)
// dp[idx+1][nxt[st][c]] += dp[idx][st];
// If avoiding pat: ensure nxt[st][c] != m
```

10.4 Suffix Array [142 lines] - abba9017

```
// for integers: change string to vector<int>, and min
// value of the vector must be >= 1
array<vector<int>, 2> get_sa(string& s, int lim=128) {
  int n = s.size() + 1, k = 0, a, b;
  vector<int> x(begin(s), end(s)+1);
  vector<int> y(n), sa(n), lcp(n);
  vector<int> ws(max(n, lim)), rank(n);
  x.back() = 0;
  iota(begin(sa), end(sa), 0);
  for (int j = 0, p = 0; p < n;
       j = max(1, j * 2), lim = p) {
    p = j, iota(begin(y), end(y), n - j);
    for (int i = 0; i < n; ++i){
      if (sa[i] >= j) y[p++] = sa[i] - j;
    }
    fill(begin(ws), end(ws), 0);
    for (int i = 0; i < n; ++i) ws[x[i]]++;
    for (int i = 1; i < lim; ++i) ws[i] += ws[i - 1];
    for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
    swap(x, y), p = 1, x[sa[0]] = 0;
    for (int i = 1; i < n; ++i){
      a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j])
        ? p - 1 : p++;
    }
  }
  for (int i = 1; i < n; ++i) rank[sa[i]] = i;
  for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k){
    for (k && k--, j = sa[rank[i] - 1];
         s[i + k] == s[j + k]; k++);
  }
  sa.erase(sa.begin()), lcp.erase(lcp.begin());
  return {sa, lcp};
}
## Comparing Two Substrings
auto compare = [&] (int l1, int r1, int l2, int r2) {
  int len1 = r1 - l1 + 1, len2 = r2 - l2 + 1;
  int len = min(len1, len2);
  // st = Sparse(sa, lcp) below; takes positions
  int x = (l1 != l2) ? st.get_lcp(l1, l2) : len;
  if (x >= len) {
    if (len1 == len2) return 0;
    if (len1 < len2) return -1;
    return 1;
  }
  if (s[l1 + x] < s[l2 + x]) return -1;
  return 1;
};
## Kth Unique Substring
auto kth = [&] (ll k) {
  int i = 0;
  while (i + 1 < n and k > n - sa[i] - lcp[i]) {
    k -= n - sa[i] - lcp[i];
    i++;
  }
  k = min(k, 0ll + n - sa[i] - lcp[i]);
  array<int, 2> ret = {sa[i], (int)(k + lcp[i])};
  return ret;
};
## Kth Unique Substring (Multiple Queries)
vector<ll> pref(n + 1);
for (int i = 0; i < n; ++i) {
  pref[i + 1] = pref[i] + (n - sa[i] - lcp[i]);
}
auto kth = [&] (ll k) {
  int i = lower_bound(pref.begin(), pref.end(), k)
          - pref.begin();
  i--;
  k -= pref[i];
  return array<int, 2>{sa[i], (int)(k + lcp[i])};
};
auto [i, len] = kth(k);
cout << s.substr(i, len) << "\n";
## Kth Substring (not necessarily unique)
ll k; cin >> k;
auto ok = [&] (ll x) {
  auto [i, len] = kth(x);
  ll tot = 0;
  for (int j = 0; j < pos[i]; ++j) {
    tot += n - sa[j];
  }
  tot += len;
  int mn = len;
  for (int j = pos[i] + 1; j < n; ++j) {
    mn = min(mn, lcp[j]);
    tot += mn;
  }
  return tot >= k;
};
ll lo = 1, hi = 1;
while (!ok(hi)) hi = hi << 1;
while (lo <= hi) {
  ll mid = (lo + hi) >> 1;
  if (ok(mid)) hi = mid - 1;
  else lo = mid + 1;
}
## Several Consecutive Identical Substrings
auto [sa, lcp] = get_sa(s);
Sparse st(sa, lcp);
auto rev_s = s;
reverse(rev_s.begin(), rev_s.end());
auto [rev_sa, rev_lcp] = get_sa(rev_s);
Sparse rev_st(rev_sa, rev_lcp);
int k = 1;
for (int len = 1; len <= n / 2; ++len) {
  for (int i = 0; i + len < n; i += len) {
    int er = st.get_lcp(i, i + len);
    int el = 0;
    // rev lcp of (i - 1, i + len - 1)
    if (i) el = rev_st.get_lcp(n - i, n - i - len);
    k = max(k, 1 + (er + el) / len);
  }
}
## Sparse Table for LCP Array
struct Sparse { // lcp of suffixes at positions l, r
  static const int K = 20; // 2^K > n
  int n;
  vector<array<int, K>> st;
  vector<int> pos;
  Sparse (vector<int> sa, vector<int> &lcp) {
    n = sa.size();
    pos.resize(n);
    for (int i = 0; i < n; ++i) {
      pos[sa[i]] = i;
    }
    st.resize(n);
    for(int i = 0; i < n; ++i) st[i][0]=lcp[i];
    for(int k = 0; k + 1 < K; ++k)
      for(int i = 0; i + (2 << k) <= n; ++i)
        st[i][k + 1] =
            min(st[i][k], st[i + (1 << k)][k]);
  }
  int get_lcp(int l, int r) {
    l = pos[l];
    r = pos[r];
    if (l > r) swap(l, r);
    assert(l < r);
    l++;
    int k = __lg(r - l + 1);
    return min(st[l][k], st[r - (1 << k) + 1][k]);
  }
};
```

10.5 Suffix Automaton [132 lines] - 9e196d73

```
// Big alphabet: `int nxt[26]` -> `map<int, int> nxt`
// & `!t[p].nxt[c]` -> `!t[p].nxt.count(c)` everywhere!
struct SAM {
  struct Node {
    int nxt[26]{}, lnk = -1, len = 0, focc = -1;
    long long cnt = 0, dp = -1;
  };
  vector<Node> t; int lst = 0; vector<int> ord;
  SAM(int n = 0) {
    t.clear(); ord.clear();
    t.reserve(2 * n + 1); t.emplace_back(); lst = 0;
  }
  void add(char ch, int idx = -1) {
    int c = ch - 'a', p = lst;
    if (t[p].nxt[c]) {
      int q = t[p].nxt[c];
      if (t[p].len + 1 == t[q].len) lst = q;
      else {
        int clone = t.size(); t.push_back(t[q]);
        t[clone].len = t[p].len + 1; t[clone].cnt = 0;
        while (p != -1 && t[p].nxt[c] == q) {
          t[p].nxt[c] = clone; p = t[p].lnk;
        }
        t[q].lnk = clone; lst = clone;
      }
      t[lst].cnt++; return;
    }
    int cur = t.size(); t.emplace_back();
    t[cur].len = t[p].len + 1; t[cur].focc = idx;
    // ONLY pure prefix nodes get a 1!
    // (Set 0 if only unique counts matter in Gen. SAM)
    t[cur].cnt = 1; lst = cur;
    while (p != -1 && !t[p].nxt[c]) {
      t[p].nxt[c] = cur; p = t[p].lnk;
    }
    if (p == -1) { t[cur].lnk = 0; return; }
    int q = t[p].nxt[c];
    if (t[p].len + 1 == t[q].len) {
      t[cur].lnk = q; return;
    }
    int clone = t.size(); t.push_back(t[q]);
    t[clone].len = t[p].len + 1;
    t[clone].cnt = 0; // Cloned nodes get 0!
    while (p != -1 && t[p].nxt[c] == q) {
      t[p].nxt[c] = clone; p = t[p].lnk;
    }
    t[q].lnk = t[cur].lnk = clone;
  }
  void build(const string& s) {
    for (int i = 0; i < s.size(); i++) add(s[i], i);
  }
  // Topo Sort + Suffix link Tree DP (freq of state)
```

```
  void count() {
    int n = t.size(); ord.resize(n);
    vector<int> c(n + 1, 0);
    for (int i = 0; i < n; i++) c[t[i].len]++;
    for (int i = 1; i <= n; i++) c[i] += c[i - 1];
    for (int i = 0; i < n; i++) ord[--c[t[i].len]] = i;
    for (int i = n - 1; i > 0; i--)
      t[t[ord[i]].lnk].cnt += t[ord[i]].cnt;
  }
  void reset_dp() { for (auto& node : t) node.dp = -1; }
  // DAG DP (paths for K-th substring / total substrs)
  // MAKE SURE to call count() first if dist == false!
  long long calc_dp(int u, bool dist) {
    if (t[u].dp != -1) return t[u].dp;
    t[u].dp = dist ? 1 : t[u].cnt; // Base case
    for (int i = 0; i < 26; i++) if (t[u].nxt[i])
      t[u].dp += calc_dp(t[u].nxt[i], dist);
    return t[u].dp;
  }
  // Find lexicographically K-th Substring
  string get_kth(long long k, bool dist) {
    if (t[0].dp == -1) calc_dp(0, dist);
    if (k > t[0].dp - (dist ? 1 : t[0].cnt))
      return "-1";
    string ans = ""; kth(0, k, ans, dist); return ans;
  }
  void kth(int u, long long& k, string& ans,
           bool dist) {
    if (k <= 0) return;
    for (int i = 0; i < 26; i++) {
      int v = t[u].nxt[i];
      if (!v) continue;
      if (k > t[v].dp) { k -= t[v].dp; continue; }
      ans += (char)('a' + i);
      k -= (dist ? 1 : t[v].cnt);
      kth(v, k, ans, dist); return;
    }
  }
};
/* --- NODE VARIABLES (BUCKETS) ---
len : Max string len in bucket (Min = t[lnk].len + 1)
focc: 0-based end pos of FIRST occ (first in endpos)
cnt : Exact Occ(VALID AFTER count()), distinct occ: 1
lnk : Suffix link (longest suffix in other bucket)
nxt : DAWG transitions (edges). dp : DAG DP memo state.
lst : Node ID of the whole string built so far
Longest at u: s.substr(t[u].focc-t[u].len+1, t[u].len)
--- 1. BUILD & GENERALIZED SAM (GSA) ---
SAM sa(s.size()); sa.build(s);
GSA: for(auto& str: strs) {sa.lst = 0; sa.build(str);}
--- 2. COUNTING & K-TH (Always call sa.count()!) ---
Distinct Subs : sa.calc_dp(0, 1) - 1 // -1: empty str
Total Subs    : sa.calc_dp(0, 0) - t[0].cnt
K-th Distinct : sa.get_kth(K, 1)
K-th Total    : sa.get_kth(K, 0)
--- 3. PATTERN MATCH (Is Substr? / Occ / First Idx) ---
int u = 0; bool ok = true; // count() first for cnt
for(char c : p) if(!(u = sa.t[u].nxt[c-'a'])))
  { ok = false; break; }
// If ok: Occurrences = sa.t[u].cnt,
// First Occ Idx (0-based) = sa.t[u].focc-p.size()+1
--- 4. LONGEST COMMON SUBSTRING (LCS of s1 & s2) ---
SAM sa(s1.size()); sa.build(s1);
int u = 0, l = 0, max_l = 0, best_pos = 0;
for(int i = 0; i < s2.size(); i++) {
  int c = s2[i] - 'a';
  while(u != -1 && !sa.t[u].nxt[c])
    { u = sa.t[u].lnk; l = u==-1 ? 0 : sa.t[u].len; }
  if(u != -1) {
    u = sa.t[u].nxt[c];
    if(++l > max_l) { max_l = l; best_pos = i; }
  } else { u = 0; l = 0; }
} // LCS = s2.substr(best_pos - max_l + 1, max_l)
--- 5. SMALLEST CYCLIC SHIFT (Lexicographically) ---
SAM sa(2 * s.size()); sa.build(s + s);
int u = 0; string ans = "";
for(int i = 0; i < s.size(); i++)
  for(int c = 0; c < 26; c++) if(sa.t[u].nxt[c])
    {ans += (char)(c+'a'); u = sa.t[u].nxt[c]; break;}
LCP of suffixes = LCS of prefixes in reversed string */
```

10.6 Trie [43 lines] - 147df034

```
struct Trie {
  static const int K = 26;
  static const char BASE = 'a';
  struct Node {
    int nxt[K]{};  // {} zero-initializes the array
    int eow = 0, cnt = 0;
  };
  vector<Node> t;
  Trie() : t(1) {} // Initializes with root at index 0
  //d = 1 for insert, d = -1 for erase
  void insert(const string& s, int d = 1) {
    int u = 0;
    t[u].cnt += d;
    for (char c : s) {
      int v = c - BASE;
      if (!t[u].nxt[v]) {
        t[u].nxt[v] = t.size();
        t.emplace_back();
      }
      u = t[u].nxt[v];
      t[u].cnt += d;
    }
    t[u].eow += d;
  }
  int search(const string& s) {
    int u = 0;
    for (char c : s) {
      int v = c - BASE;
      if (!t[u].nxt[v]) return 0;
      u = t[u].nxt[v];
    }
    return t[u].eow;
  }
  int count_prefix(const string& s) {
    int u = 0;
    for (char c : s) {
      int v = c - BASE;
      if (!t[u].nxt[v]) return 0;
      u = t[u].nxt[v];
    }
    return t[u].cnt;
  }
};
```

10.7 Z-function and Manacher [36 lines] - c59c0f88

```
// z[i] = longest common prefix of s and s[i..], z[0]=0
// search: z of (p + '#' + t), match where z == |p|
vector<int> zfunc(const string& s) {
  int n = s.size();
  vector<int> z(n, 0);
  for (int i = 1, l = 0, r = 0; i < n; i++) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]])
      z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
  }
  return z;
}
// Manacher: d1[i] = # odd palindromes centered at i
// (longest = 2*d1[i]-1), d2[i] = # even palindromes
// centered between i-1 and i (longest = 2*d2[i]). O(n)
pair<vector<int>, vector<int>> manacher(string& s) {
  int n = s.size();
  vector<int> d1(n), d2(n);
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
    while (i - k >= 0 && i + k < n &&
           s[i - k] == s[i + k]) k++;
    d1[i] = k--;
    if (i + k > r) l = i - k, r = i + k;
  }
  for (int i = 0, l = 0, r = -1; i < n; i++) {
    int k = (i > r) ? 0
                    : min(d2[l + r - i + 1], r - i + 1);
    while (i - k - 1 >= 0 && i + k < n &&
           s[i - k - 1] == s[i + k]) k++;
    d2[i] = k--;
    if (i + k > r) l = i - k - 1, r = i + k;
  }
  return {d1, d2};
}
```

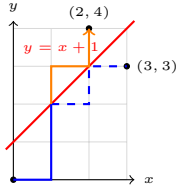
11 Random

11.1 Combinatorics

- $\sum_{k=0}^{n} \binom{n-k}{k} = Fib_{n+1}$
- $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
- $k\binom{n}{k} = n\binom{n-1}{k-1}$
- Number of binary sequences of length n such that no two 0's are adjacent $= Fib_{n+1}$
- Number of non-negative solution of $x_1 + x_2 + x_3 + ... + x_k = n$ is $\binom{n+k-1}{n}$

11.1.1 Catalan Number

- **Reflection Principle** $(n = 3)$
  Count paths $(0,0) \to (n,n)$ *below* $y = x$.



  - **Bad Path:** Touches $y = x + 1$.

  - **Reflection:** Flip path after first touch.
  - **Mapping:** Bad path to $(n, n) \iff$ Path to $(n-1, n+1)$.
- $C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$
- $C_0 = 1, C_1 = 1, C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$
- $1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786$
- Number of correct bracket sequences consisting of n opening brackets.
- Number of ways to completely parenthesize n+1 factors.
- The number of triangulations of a convex polygon with +2 sides (i.e. the number of partitions of polygon into disjoint triangles by using the diagonals).
- The number of ways to connect the 2n points on a circle to form n disjoint i.e. non-intersecting chords.
- The number of monotonic lattice paths from point (0,0) to point (n,n) in a square lattice of size $n \times n$, which do not pass above the main diagonal
- Number of permutation of length n that can be stack sorted.
- The number of non-crossing partitions of a set of n elements.
- The number of rooted full binary tree with n+1 leaves.
- The number of Dyck words of length 2n. A string consisting of n X's and n Y's such that no string prefix has more Y's than X's.
- Number of permutation of length n with no three-term increasing subsequence.
- Number of ways to tile a stairstep shape of height n with n rectangle.
- $C_n^k = \frac{k+1}{n+1}\binom{2n-k}{n-k}$ denote the number of bracket sequences of size 2n with the first k elements being (.
- $N(n, k) = \frac{1}{n}\binom{n}{k}\binom{n}{k-1}$
- The number of expressions containing n pairs of correct parentheses, which contain k distinct nestings. $N(4, 2) = 6$
  $()(((\,))), (())((\,)), (()((\,))), (((\,))()), ((()())), (((\,)))()$
- The number of paths from (0,0) to (2n, 0) with steps only northeast and southeast, not staying below the x-axis with k peaks. And sum of all number of peaks is Catalan number.

11.1.2 Stirling Number of the First Kind

- Count permutation according to their number of cycles.
- $S(n, k)$ count the number of permutation of n elements with k disjoint cycles.
- $S(n, k) = (n-1) \times S(n-1, k) + S(n-1, k-1), S(0, 0) = 1, S(n, 0) = S(0, n) = 0$
- $S(n, 1) = (n-1)!$
- $S(n, n-1) = \binom{n}{2}$
- $\sum_{k=0}^{n} S(n, k) = n!$

11.1.3 Stirling Numbers of the Second Kind

- Number of ways to partition a set of n objects into k non-empty subsets.
- $S(n, k) = k * S(n-1, k) + S(n-1, k-1), S(0, 0) = 1, S(n, 0) = S(0, n) = 0$
- $S(n, 2) = 2^{n-1} - 1$
- $S(n, k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$
- $S(n, k) * k!$ = number of ways to color n nodes using colors from 1 to k such that each color is used at least once.

11.1.4 Bell Number

- Counts the number of partitions of a set.
- $B_{n+1} = \sum_{k=0}^{n}\binom{n}{k} * B_k$
- $B_n = \sum_{k=0}^{n} S(n, k)$, where S is Stirling number of second kind.
- The number of multiplicative partitions of a square free number with i prime factors is the i-th Bell number.
- $B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$
- If a deck is shuffled by removing and reinserting the top card n times, there are n^n possible shuffles. The number of shuffles that return the deck to its original order is B_n, so the probability of returning to the original order is B_n/n^n.

11.1.5 Lucas Theorem

- If p is prime then $\binom{p^a}{k} \equiv 0 \pmod{p}$
- For non-negative integers m and n and a prime p:
  $\binom{m}{n} = \prod_{i=0}^{k}\binom{m_i}{n_i} \pmod{p}$ where
  $m = m_k p^k + m_{k-1}p^{k-1} + ... + m_1 p + m_0$ $n = n_k p^k + n_{k-1}p^{k-1} + ... + n_1 p + n_0$ are the base p expansion.
- Let n and k be non-negative integers, and let p be a prime. Let the base-p expansions of n and k be:

$$n = n_m p^m + n_{m-1}p^{m-1} + \cdots + n_1 p^1 + n_0 p^0$$

$$k = k_m p^m + k_{m-1}p^{m-1} + \cdots + k_1 p^1 + k_0 p^0$$

  where $0 \le n_i < p$ and $0 \le k_i < p$ are the base-p digits of n and k, respectively.

  The theorem states:

$$\binom{n}{k} \equiv \prod_{i=0}^{m}\binom{n_i}{k_i} \pmod{p}$$

  Note: The individual terms $\binom{n_i}{k_i}$ are calculated using standard arithmetic, and the final product is taken modulo p.

- **Example:** $\binom{10}{2} \pmod 2$

$$n = 10_{10} = 1010_2$$
$$k = 2_{10} = 0010_2$$

  Applying Lucas's Theorem:

$$\binom{10}{2} \equiv \binom{1}{0} \cdot \binom{0}{0} \cdot \binom{1}{1} \cdot \binom{0}{0} \pmod 2$$

$$\equiv 1 \cdot 1 \cdot 1 \cdot 1 \equiv 1 \pmod 2$$

  Bitwise check: $1010_2 \& 0010_2 = 0010_2$. Since $2 = 2$, the result is 1.

$$\binom{n}{k} \pmod 2 = \begin{cases} 1 & \text{if } (n \& k) = k \\ 0 & \text{otherwise} \end{cases}$$

11.1.6 Derangement

- A permutation such that no element appears in its original position.
- $d(n) = (n-1) * (d(n-1) + d(n-2)), d(0) = 1, d(1) = 0$
- $d(n) = nd(n-1) + (-1)^n = \lfloor \frac{n!}{e} \rfloor, n \ge 1$

11.1.7 Burnside Lemma

Given a group G of symmetries and a set X, the number of elements of X up to symmetry equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|$$

where X^g are the elements fixed by $g(g.x = x)$ If f(n) counts "configurations" of some sort of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k)$$

11.1.8 Eulerian Number

- $E(n,k)$ is the number of permutations of the numbers 1 to n in which exactly k elements are greater than the previous element.
- $E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k), E(n,0) = E(n,n-1) = 1$
- $E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$
- $E(n,k) = E(n,n-1-k)$
- $E(0,k) = [k=0]$
- $E(n,1) = 2^n - n - 1$

11.2 Number Theory

11.2.1 Mobius Function and Inversion

- define $\mu(n)$ as the sum of the primitive nth roots of unity depending on the factorization of n into prime factors:

$$\mu(x) = \begin{cases} 0 & \text{n is not square free} \\ 1 & \text{n has even number of prime factors} \\ -1 & \text{n has odd number of prime factors} \end{cases}$$

- Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \leftrightarrow f(n) = \sum_{d|n}\mu(d)g(n/d)$$

- $\sum_{d|n}\mu(d) = [n=1]$
- $\phi(n) = \sum_{d|n}\mu(d).\frac{n}{d} = n\sum_{d|n}\frac{\mu(d)}{d} = \sum_{d|n} d.\mu(\frac{n}{d})$
- $a|b \to \phi(a)|\phi(b)$
- $\phi(mn) = \phi(m).\phi(n).\frac{d}{\phi(d)}$ where $d = gcd(m,n)$
- $\phi(n^m) = n^{m-1}\phi(n)$
- $\sum_{i=1}^{n}[gcd(i,n)=k] = \phi(\frac{n}{k})$
- $\sum_{i=1}^{n} gcd(i,n) = \sum_{d|n} d.\phi(\frac{n}{d})$
- $\sum_{i=1}^{n}\frac{1}{gcd(i,n)} = \sum_{d|n}\frac{1}{d}.\phi(\frac{n}{d}) = \frac{1}{n}\sum_{d|n} d.\phi(d)$
- $\sum_{i=1}^{n}\frac{i}{gcd(i,n)} = \frac{n}{2}.\sum_{d|n}\frac{1}{d}.\phi(\frac{n}{d}) = \frac{n}{2}.\frac{1}{n}\sum_{d|n} d.\phi(d)$
- $\sum_{i=1}^{n}\frac{n}{gcd(i,n)} = 2.\sum_{i=1}^{n}\frac{i}{gcd(i,n)} - 1$

11.2.2 GCD and LCM

- gcd(a,b) = gcd(b, a mod b)
- If $a|b.c$, and gcd(a,b) = d, then $(a/d)|c$.
- GCD is a multiplicative function.
- gcd(a, lcm(b,c)) = lcm(gcd(a,b), gcd(a,c))
- $gcd(n^a-1, n^b-1) = n^{gcd(a,b)} - 1$

11.2.3 Gauss Circle Theorem

- Determine the number of lattice points in a circle centered at the origin with radius r.
- number of pairs (m,n) such that $m^2 + n^2 \leq r^2$
- $N(r) = 1 + 4\sum_{i=0}^{\infty}(\lfloor\frac{r^2}{4i+1}\rfloor - \lfloor\frac{r^2}{4i+3}\rfloor)$

11.2.4 Pick's Theorem

According to Pick's Theorem We can calculate the area of any polygon by just counting the number of Interior and Boundary lattice points of that polygon. If number of interior points are I and number of boundary lattice points are B then Area (A) of polygon will be:

$$Area = I + B/2 - 1$$

where I is the number of points in the interior shape, B stands for the number of points on the boundary of the shape.

11.2.5 Formula Cheatsheet

- $\sum_{i=1}^{n} = \frac{1}{m+1}[(n+1)^{m+1} - 1 - \sum_{i=1}^{n}((i+1)^{m+1} - i^{m+1} - (m+1)i^m)]$
- $\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}, c \neq 1$
- $\sum_{i=0}^{\infty} c^i = \frac{1}{1-c}, \sum_{i=1}^{\infty} c^i = \frac{c}{1-c}, |c| < 1$
- $H_n = \sum_{i=1}^{n}\frac{1}{n}, \sum_{i=1}^{n} iH_i = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}$
- $\sum_{k=0}^{n}\binom{r+k}{k} = \binom{r+n+1}{n}$

11.2.6 Geometry and Formulas

- **Area Formulas**
  - Rectangle: $A = l \times w$
  - Square: $A = s^2$
  - Triangle: $A = \frac{1}{2}bh$
  - Circle: $A = \pi r^2$
  - Parallelogram: $A = bh$
  - Pyramid Base: $A = \frac{1}{2}b \times h_{slant}$
  - Polygon: $A = \frac{1}{2}|\sum_{i=1}^{n-1}(x_i y_{i+1})|$
  - Pick's formula: $A = I + \frac{B}{2} - 1$ where I = interior points, B = boundary points
- **Perimeter Formulas**
  - Rectangle: $P = 2(l+w)$
  - Square: $P = 4s$
  - Triangle: $P = a + b + c$
  - Circle: $P = 2\pi r$
- **Volume Formulas**
  - Cube: $V = s^3$
  - Rect. Prism: $V = lwh$
  - Cylinder: $V = \pi r^2 h$
  - Sphere: $V = \frac{4}{3}\pi r^3$
  - Pyramid: $V = \frac{1}{3}A_{base}h$
- **Surface Area Formulas**
  - Cube: $SA = 6s^2$

- Rect. Prism: $SA = 2(lw + lh + wh)$
- Cylinder: $SA = 2\pi r(r + h)$
- Sphere: $SA = 4\pi r^2$
- Pyramid: $SA = A_{base} + \frac{1}{2}P_{base}h_{slant}$

- **Triangles**
  - Side Lengths: a, b, c
  - Semi Perimeter: $p = \frac{a+b+c}{2}$
  - Area (Heron's): $A = \sqrt{p(p-a)(p-b)(p-c)}$
  - Circumradius: $R = \frac{abc}{4A}$
  - Inradius: $r = \frac{A}{p}$

- **Summation Of Series**
  - $c^k + c^{k+1} + \cdots + c^n = \frac{c^{n+1}-c^k}{c-1}$
  - $1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$
  - $1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$
  - $1^3 + 2^3 + 3^3 + \cdots + n^3 = \left(\frac{n(n+1)}{2}\right)^2$
  - $1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$
  - **Arithmetic**: $S_n = \frac{n}{2}\left[2a + (n-1)d\right]$
  - **Geometric**: $S_n = a \cdot \frac{r^n-1}{r-1}$

- **Triangle Laws**
  - **Law of Sines**: $\dfrac{a}{\sin A} = \dfrac{b}{\sin B} = \dfrac{c}{\sin C} = 2R$
  - **Law of Cosines**: $a^2 = b^2 + c^2 - 2bc\cos A$
  - **Law of Tangents**: $\dfrac{a+b}{a-b} = \dfrac{\tan\left(\frac{A+B}{2}\right)}{\tan\left(\frac{A-B}{2}\right)}$

- **Angle Sum and Difference**

$$\sin(A+B) = \sin A\cos B + \cos A\sin B$$
$$\cos(A+B) = \cos A\cos B - \sin A\sin B$$
$$\sin(A-B) = \sin A\cos B - \cos A\sin B$$
$$\cos(A-B) = \cos A\cos B + \sin A\sin B$$
$$\tan(A+B) = \frac{\tan A + \tan B}{1 - \tan A\tan B}$$
$$\tan(A-B) = \frac{\tan A - \tan B}{1 + \tan A\tan B}$$

- **Double Angle Formulas**

$$\sin 2\theta = 2\sin\theta\cos\theta$$
$$\cos 2\theta = \cos^2\theta - \sin^2\theta$$
$$\tan 2\theta = \frac{2\tan\theta}{1 - \tan^2\theta}$$

- **Half Angle Formulas**

$$\sin\left(\frac{\theta}{2}\right) = \pm\sqrt{\frac{1-\cos\theta}{2}}$$
$$\cos\left(\frac{\theta}{2}\right) = \pm\sqrt{\frac{1+\cos\theta}{2}}$$
$$\tan\left(\frac{\theta}{2}\right) = \frac{1-\cos\theta}{\sin\theta}$$

- **Sum-to-Product Formulas**

$$\sin r + \sin w = 2\sin\left(\frac{r+w}{2}\right)\cos\left(\frac{r-w}{2}\right)$$
$$\cos r + \cos w = 2\cos\left(\frac{r+w}{2}\right)\cos\left(\frac{r-w}{2}\right)$$
$$(V+W)\tan\left(\frac{r-w}{2}\right) = (V-W)\tan\left(\frac{r+w}{2}\right)$$

- **Linear Combinations**

$$a\cos x + b\sin x = r\cos(x-\phi)$$
$$a\sin x - b\cos x = r\sin(x-\phi)$$
$$\text{where } r = \sqrt{a^2+b^2}, \quad \phi = \arctan(b/a)$$

11.2.7 Most Number of Divisors

Max Value	Number	Divisors
10^3	83,160	128
10^6	720,720	240
10^7	9,609,600	640
10^8	98,280,000	672
10^9	735,134,400	1,344
10^{10}	7,242,460,800	2,688
10^{11}	73,346,256,000	5,376
10^{12}	936,966,912,400	10,752

11.2.8 Pascal's Triangle

$$\begin{array}{ccccccccccc}
 & & & & & 1 & & & & & \\
 & & & & 1 & & 1 & & & & \\
 & & & 1 & & 2 & & 1 & & & \\
 & & 1 & & 3 & & 3 & & 1 & & \\
 & 1 & & 4 & & 6 & & 4 & & 1 & \\
1 & & 5 & & 10 & & 10 & & 5 & & 1
\end{array}$$

1	6	15	20	15	6	1				
1	7	21	35	35	21	7	1			
1	8	28	56	70	56	28	8	1		
1	9	36	84	126	126	84	36	9	1	
1	10	45	120	210	252	210	120	45	10	1

11.2.9 Logarithm Rules

- $\log_b(xy) = \log_b x + \log_b y$
- $\log_b\left(\dfrac{x}{y}\right) = \log_b x - \log_b y$
- $\log_b(x^k) = k\log_b x$
- $\log_b b = 1$
- $\log_b 1 = 0$
- $\log_b x = \dfrac{\log_k x}{\log_k b}$

11.2.10 Miscellaneous

- $2^{100} = 2^{50} \times 2^{50}$
- Fibonacci Matrix:

$$\begin{bmatrix} F_n \\ F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} F_1 \\ F_0 \end{bmatrix}$$

- $\log(n!) = \log 1 + \log 2 + \cdots + \log n$
- $\gcd(a, b) = \gcd(a - b, b)$
- Prime p in $n!$:

$$\left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \left\lfloor \frac{n}{p^3} \right\rfloor + \cdots$$

- Catalan:

$$\binom{2n}{n} - \binom{2n}{n-1}$$

- Divisors of $p^x q^y$: $(x+1)(y+1)$
- Sum of divisors of $p^x q^y$:

$$(1 + p + p^2 + \cdots + p^x)(1 + q + q^2 + \cdots + q^y)$$

- Divisibility rules:
  - **3**: Sum of digits divisible by 3
  - **4**: Last two digits divisible by 4
  - **5**: Last digit 0 or 5
  - **6**: Divisible by both 2 and 3
  - **7**: Double last digit, subtract from rest. Continue till manageable.
  - **8**: Last three digits divisible by 8
  - **9**: Sum of digits divisible by 9
  - **11**: $(sum\ of\ odd\ digits) - (sum\ of\ even\ digits) \equiv 0 \pmod{11}$
- Derangement: $D_n = (n-1)D_{n-2} + (n-1)D_{n-1}$
- Golden Ratio: $\Phi = \frac{1+\sqrt{5}}{2} \approx 1.618034$
- n-th Fibonacci:

$$F_n = \frac{\Phi^n - (1-\Phi)^n}{\sqrt{5}}$$

- Stars & Bars: Solutions of $x_1 + x_2 + \cdots + x_k = n$ $x_i > 0$: $\binom{n-1}{k-1}$, $x_i \geq 0$: $\binom{n+k-1}{k-1}$
- **Throwing Dice (Matrix Exponentiation)**
  - **Recurrence Relation** The number of ways to achieve a sum n is given by the linear recurrence:

$$\begin{aligned} f(n) =& f(n-1) + f(n-2) + f(n-3) \\ &+ f(n-4) + f(n-5) + f(n-6) \end{aligned}$$

    with the base case $f(0) = 1$ and $f(k) = 0$ for $k < 0$.
  - **State Transition** The relationship between the new state vector (V_i) and the old state vector (V_{i-1}) is:

$$\underbrace{\begin{bmatrix} f(i) \\ f(i-1) \\ f(i-2) \\ f(i-3) \\ f(i-4) \\ f(i-5) \end{bmatrix}}_{V_i} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}}_{T} \times \underbrace{\begin{bmatrix} f(i-1) \\ f(i-2) \\ f(i-3) \\ f(i-4) \\ f(i-5) \\ f(i-6) \end{bmatrix}}_{V_{i-1}}$$

  - **Final Answer** The n-th term is found by: $f(n) = (T^n)_{1,1} \times f(0)$, where $f(0) = 1$.

$$f(n) = (T^n)_{1,1}$$

11.3 Probability and Expectation

- Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.
- Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

  For independent X and Y,

$$V(aX + bY) = a^2 V(X) + b^2 V(Y).$$

- **Discrete distributions**
  - **Binomial distribution:** The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n, p)$, $n = 1, 2, \ldots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

    $\text{Bin}(n, p)$ is approximately $\text{Po}(np)$ for small p.
  - **First success distribution:** The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\text{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

  - **Poisson distribution:** The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\text{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \ldots$$

$$\mu = \lambda,\ \sigma^2 = \lambda$$

- **Continuous distributions**
  - **Uniform distribution:** If the probability density function is constant between a and b and 0 elsewhere it is $\text{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b-a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a+b}{2},\ \sigma^2 = \frac{(b-a)^2}{12}$$

  - **Exponential distribution:** The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda},\ \sigma^2 = \frac{1}{\lambda^2}$$

  - **Normal distribution:** Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

    If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$